UNIVERSIDADE FEDERAL DE ALAGOAS
INSTITUTO DE COMPUTAÇÃO
CURSO DE CIÊNCIA DA COMPUTAÇÃO

ANTHONY GABRIEL PAULINO DE MOURA
LUCAS TEODOSIO MACIEL DE OLIVEIRA
PEDRO CABRAL GOMES
VINICIUS GABRIEL SILVA SANTIAGO

DESENVOLVIMENTO DE UM COMPILADOR PARA A LINGUAGEM MINIPAR 2026.2 ORIENTADA A OBJETOS UTILIZANDO REAL PARALELISMO COM EXECUÇÃO EM COMPUTADORES DIFERENTES

uso de Fork e Join e implementação com os canais de comunicação (c_channel) do MiniPar

MACEIÓ – AL
2026

ANTHONY GABRIEL PAULINO DE MOURA
LUCAS TEODOSIO MACIEL DE OLIVEIRA
PEDRO CABRAL GOMES
VINICIUS GABRIEL SILVA SANTIAGO

DESENVOLVIMENTO DE UM COMPILADOR PARA A LINGUAGEM MINIPAR 2026.2 ORIENTADA A OBJETOS UTILIZANDO REAL PARALELISMO COM EXECUÇÃO EM COMPUTADORES DIFERENTES

uso de Fork e Join e implementação com os canais de comunicação (c_channel) do MiniPar

Relatório técnico do projeto do compilador MiniPar 2026.2, apresentado à disciplina de Compiladores do Curso de Ciência da Computação do Instituto de Computação da Universidade Federal de Alagoas, como requisito parcial para avaliação.

Professor: Arturo Hernandez Dominguez

Anthony Gabriel Paulino de Moura	202407213
Lucas Teodosio Maciel de Oliveira	202433572
Pedro Cabral Gomes	202407410
Vinicius Gabriel Silva Santiago	202407222

Repositório: `https://github.com/anthonygpm/Projeto-MiniPar-2026.2`

MACEIÓ – AL
2026

RESUMO

Este relatório descreve o compilador da linguagem MiniPar 2026.2, construído sobre a versão 2026.1 do projeto. O compilador, escrito em Python, passa o programa por análise léxica, análise sintática descendente recursiva, análise semântica com tabela de símbolos, geração de código de três endereços e geração de C++20, compilado com g++ em um executável nativo. A principal novidade da versão é o paralelismo explícito com Fork e Join: a construção `fork f(x) on core(n)` executa uma função em um núcleo escolhido da CPU, `on host(ip, porta)` envia a chamada a outro computador que roda o mesmo programa em modo worker, e `join` espera a subtarefa e devolve o resultado. O fork em outro computador foi validado entre duas máquinas reais na mesma rede. A linguagem também ganhou `null`, concatenação de texto com número e a função `write_file`, necessárias para o Teste 11, um minicompilador de expressões escrito em MiniPar que gera Assembly ARMv7, executado no CPULATOR. O projeto inclui uma interface Web didática que mostra tokens, árvore sintática, tabela de símbolos, código intermediário e C++ de cada programa, e uma suíte de 64 testes automatizados de integração. Nos Testes 8, 9 e 10, as versões paralelas obtiveram speedup entre 1,6 e 3,4 vezes, conforme a máquina.

Palavras-chave: compiladores; MiniPar; paralelismo; fork e join; computação distribuída.

SUMÁRIO

1 INTRODUÇÃO **5**

2 A LINGUAGEM MINIPAR 2026.2 . . . . . **5**

2.1 Novidades da versão 2026.2 6

2.2 Produções acrescentadas à gramática 7

2.3 Gramática BNF completa 7

3 ANALISADOR LÉXICO **9**

4 ANALISADOR SINTÁTICO **11**

5 ANALISADOR SEMÂNTICO E TRATAMENTO DE ERROS **14**

5.1 Regras verificadas 14

5.2 Pseudocódigo 15

5.3 Tratamento de erros em cada fase 17

6 TABELA DE SÍMBOLOS **17**

7 INTERFACES DO COMPILADOR **19**

7.1 Terminal 19

7.2 Web 1: interface do desenvolvedor 21

7.3 Web 2: interface didática (nova em 2026.2) 23

8 PRODUCT BACKLOG **29**

9 SPRINTS BACKLOG **30**

9.1 Sprint 1: base e front end 30

9.2 Sprint 2: paralelismo, interface e Teste 11 31

9.3 Sprint 3: relatório e apresentação 31

10 DIAGRAMA DE CASOS DE USO **32**

11 ARQUITETURA COM COMPONENTES DE SOFTWARE **33**

12 MODELAGEM DOS COMPONENTES: DIAGRAMA DE CLASSES **35**

12.1 Nós da AST acrescentados na versão 2026.2 36

12.2 Classes do runtime C++ 37

13 TESTES DE INTEGRAÇÃO DOS COMPONENTES **37**

13.1 Defeitos encontrados pelos testes 38

14 TESTES DOS PROGRAMAS DE EXEMPLO 1 A 11 . . . . . **39**

14.1 Teste 1: calculadora cliente-servidor 40

14.2 Teste 2: fatorial e Fibonacci em paralelo 40

14.3 Teste 3: neurônio simples (início e fim) 40

14.4 Teste 4: rede neural XOR 41

14.5 Teste 5: recomendação de produtos 41

14.6 Teste 6: quicksort com classes (entrada `10 -3 -40 80 70 -100`) 41

14.7 Teste 7: quicksort iterativo 41

14.8 Testes 8, 9 e 10: benchmark sequencial × paralelo 41

14.9 Teste 11: minicompilador de expressões para ARMv7 42

15 IMPLEMENTAÇÃO **44**

15.1 Tecnologias e ambiente 44

15.2 Passo a passo das execuções 45

15.3 Problemas encontrados e decisões 47

16 CÓDIGO-FONTE, TESTES E RELATÓRIO NO GITHUB 47
17 CONCLUSÃO 48
REFERÊNCIAS 49

1 INTRODUÇÃO

O projeto entrega um compilador para a linguagem MiniPar 2026.2 que vai do código-fonte ao executável nativo e executa programas com paralelismo real: subtarefas em núcleos diferentes da CPU ou em outro computador. Ele foi construído sobre o compilador MiniPar 2026.1 (repositório minipar-improvement), reaproveitando seus componentes, como pede o enunciado.

O compilador é escrito em Python e orientado a objetos. Cada fase é um componente de software com interface própria:

1. **Analisador léxico** — transforma o texto em tokens.
2. **Analisador sintático** — constrói a árvore sintática abstrata (AST).
3. **Analisador semântico** — confere declarações, tipos, aridade e as regras de Fork/Join, usando a tabela de símbolos.
4. **Gerador de código intermediário** — produz código de três endereços (TAC).
5. **Gerador de código C++20** — traduz a AST e chama o GCC com `-O3` para gerar o executável.

As novidades exigidas para 2026.2 e implementadas são:

- **Fork e Join** na gramática, com o destino de cada subtarefa escolhido na própria produção: `on core(n)` para um núcleo da CPU ou `on host(ip, porta)` para outro computador.
- **Testes 8, 9 e 10** (algoritmos paralelos com 4, 5 e 6 tarefas) e **Teste 11** (um minicompilador de expressões escrito em MiniPar que gera Assembly ARMv7 para o CPULATOR).
- **Segunda interface Web, didática**, mantendo o terminal e a interface Web do 2026.1.
- **Testes de integração automatizados**: 64 testes que compilam e executam todos os programas e comparam as saídas.

2 A LINGUAGEM MINIPAR 2026.2

MiniPar é uma linguagem imperativa e orientada a objetos, com tipagem dinâmica opcional (variáveis com `var` ou com tipo explícito `number`, `int`, `string`, `bool`, `list`, `dict`), escrita para ensinar paralelismo e comunicação entre processos. Um programa é uma sequência de declarações globais (classes, funções e variáveis) e comandos, organizados em blocos sequenciais (`seq { … }`) ou paralelos (`par { … }`). A comunicação entre computadores usa canais TCP (`c_channel`, `send`, `receive`). Comentários começam com `#` e vão até o fim da linha.

Principais características:

- **Blocos `seq` e `par`**: `par` executa cada comando do bloco em uma thread própria e espera todos terminarem.
- **Orientação a objetos**: `class`, `extends` (herança simples), `new`, `this`, atributos e

métodos.

- **Coleções**: listas (`[a, b, c]`, indexação e `len`) e `matrix(linhas, colunas, valor)`.
- **Canais**: `c_channel(ip, porta)` cria um canal TCP; `send(canal, valor)` e `receive(canal)` trocam mensagens entre programas que rodam em computadores diferentes.
- **Entrada e saída**: `print(…)`, `input(…)` e, na versão 2026.2, `write_file(caminho, texto)`.

2.1 Novidades da versão 2026.2

Quadro 1 – Novidades da versão 2026.2 da linguagem MiniPar

Recurso	Sintaxe	O que faz
Fork em núcleo	`var t = fork f(x) on core(2);`	Executa `f(x)` em uma thread fixada no núcleo 2 da CPU e devolve uma tarefa
Fork em outro computador	`var t = fork f(x) on host("192.168.0.10", 5001);`	Envia a chamada para um worker MiniPar em outro computador e devolve uma tarefa
Fork sem destino	`var t = fork f(x);`	Executa `f(x)` em uma thread escolhida pelo sistema operacional
Join	`var r = join t;`	Espera a tarefa `t` terminar e devolve o valor que `f` retornou
Forks em coleções	`var ts = [fork f(1), fork f(2)];`	Cria várias tarefas de uma vez; `join ts[i]` espera cada uma
`null`	`var prox = null;`	Representa a ausência de objeto (necessário para listas ligadas e árvores)
Concatenação texto + número	`print("total: " + n);`	`+` converte o número para texto quando um dos lados é texto
`write_file`	`write_file("saida.arm", codigo);`	Grava um texto em arquivo (usado pelo Teste 11 para gerar o assembly ARMv7)
Divisão real	`7 / 2` resulta em `3.5`	`/` sempre faz divisão real; divisão por zero gera erro em tempo de execução

Fonte: Elaborado pelos autores (2026).

O destino `host` exige que o outro computador rode o mesmo programa compilado em modo worker (`programa.exe --worker 5001`). A tarefa devolvida por `fork` é tratada da mesma forma em todos os destinos, então `join` não precisa saber onde a função foi executada.

2.2 Produções acrescentadas à gramática

```
<expressao_fork>  ::= "fork" <chamada> [ "on" <destino_fork> ]
<chamada>         ::= <chamada_funcao> | <chamada_metodo>
<destino_fork>    ::= "core" "(" <expressao> ")"
                    | "host" "(" <expressao> "," <expressao> ")"
<expressao_join>  ::= "join" <expressao_unaria>

<expressao>        ::= <expressao_fork> | <expressao_ou>
<expressao_unaria> ::= ( "-" | "+" | "!" ) <expressao_unaria>
                     | "join" <expressao_unaria>
                     | <expressao_posfixo>
<expressao_primaria> ::= … | "null" | …
```

`fork` aparece no nível mais alto de expressão, porque devolve uma tarefa inteira e não participa de contas. `join` é um operador unário prefixo, então `join t + 1` significa `(join t) + 1`. As novas palavras reservadas são `fork`, `join`, `on`, `core`, `host` e `null`.

2.3 Gramática BNF completa

```
(* ── ESTRUTURA GLOBAL ── *)
<programa> ::= { <declaracao_global> |
↪  <comando> }
<declaracao_global> ::=
↪  <declaracao_classe> |
↪  <declaracao_funcao> |
↪  <declaracao_variavel>

(* ── DECLARAÇÕES ── *)
<declaracao_variavel> ::= "var"
↪  <IDENTIFIER> [ "=" <expressao> ] ";"
                        | <tipo>
                        ↪  <IDENTIFIER> [
                        ↪  "="
                        ↪  <expressao> ]
                        ↪  ";"
<declaracao_funcao>   ::= "func"
↪  <IDENTIFIER> "(" [ <lista_parametros>
↪  ] ")" <bloco_seq>
<lista_parametros>    ::= <parametro> {
↪  "," <parametro> }
<parametro>           ::= [ <tipo> ]
↪  <IDENTIFIER>
<declaracao_classe>   ::= "class"
↪  <IDENTIFIER> [ "extends" <IDENTIFIER>
↪  ] "{" { <membro_classe> } "}"
<membro_classe>       ::= "var"
↪  <declaracao_variavel> | "func"
↪  <declaracao_funcao>

(* ── COMANDOS ── *)
```

```
<comando> ::= <declaracao_variavel> |
↪  <atribuicao> | <bloco_seq> |
↪  <bloco_par>
            | <comando_if> |
            ↪  <comando_while> |
            ↪  <comando_do_while> |
            ↪  <comando_for>
            | <comando_print> |
            ↪  <comando_send> |
            ↪  <comando_return>
            | <comando_break> |
            ↪  <comando_continue>
            | <chamada_funcao> ";" |
            ↪  <chamada_metodo> ";"
            | <expressao_fork> ";" |
            ↪  <expressao_join> ";"
<atribuicao>       ::= <alvo_atribuicao>
↪  "=" <expressao> ";"
<alvo_atribuicao>  ::= <IDENTIFIER>
                     | <expressao_posfix
                     ↪  o> "."
                     ↪  <IDENTIFIER>
                     | <expressao_posfix
                     ↪  o> "["
                     ↪  <expressao> "]"
<comando_print>    ::= "print" "(" [
↪  <expressao> { "," <expressao> } ] ")"
↪  ";"
<comando_send>     ::= "send" "("
↪  <expressao> "," <expressao> ")" ";"
<comando_return>   ::= "return" [
↪  <expressao> ] ";"
```

```
<comando_break>    ::= "break" ";"
<comando_continue> ::= "continue" ";"

(* ——— BLOCOS ——— *)
<bloco_seq> ::= "seq" "{" { <comando> }
↪  "}" | "{" { <comando> } "}"
<bloco_par> ::= "par" "{" { <comando> }
↪  "}"

(* ——— ESTRUTURAS DE CONTROLE ——— *)
<comando_if>        ::= "if" "("
↪  <expressao> ")" <comando> [ "else"
↪  <comando> ]
<comando_while>     ::= "while" "("
↪  <expressao> ")" <comando>
<comando_do_while>  ::= "do" <comando>
↪  "while" "(" <expressao> ")" [ ";" ]
<comando_for>       ::= "for" "("
↪  <for_cabecalho> ")" <comando>
<for_cabecalho>     ::= <for_classico> |
↪  <for_iterador>
<for_classico>      ::= [
↪  <inicializador_for> ] ";" [
↪  <expressao> ] ";" [ <incremento_for> ]
<inicializador_for> ::= "var"
↪  <IDENTIFIER> [ "=" <expressao> ]
                      | <tipo>
                      ↪  <IDENTIFIER> [
                      ↪  "=" <expressao>
                      ↪  ]
                      | <alvo_atribuicao>
                      ↪  "=" <expressao>
                      | <expressao>
<incremento_for>    ::= <alvo_atribuicao>
↪  "=" <expressao> | <expressao>
<for_iterador>      ::= "var"
↪  <IDENTIFIER> "in" <expressao>

(* ——— ORIENTAÇÃO A OBJETOS ——— *)
<expressao_new>      ::= "new"
↪  <IDENTIFIER> "(" [ <lista_argumentos>
↪  ] ")"
<acesso_propriedade> ::=
↪  <expressao_posfixo> "." <IDENTIFIER>
<chamada_metodo>     ::=
↪  <expressao_posfixo> "." <IDENTIFIER>
↪  "(" [ <lista_argumentos> ] ")"
<expressao_this>     ::= "this"

(* ——— CANAIS ——— *)
<expressao_c_channel> ::= "c_channel" "("
↪  <expressao> "," <expressao> ")"
<expressao_receive>   ::= "receive" "("
↪  <expressao> ")"

(* ——— COLEÇÕES ——— *)
<expressao_lista>  ::= "[" [ <expressao> {
↪  "," <expressao> } ] "]"
<expressao_indice> ::=
↪  <expressao_posfixo> "[" <expressao>
↪  "]"
<expressao_matrix> ::= "matrix" "("
↪  <expressao> "," <expressao> ","
↪  <expressao> ")"

(* ——— ENTRADA ——— *)
<expressao_input> ::= "input" "(" [
↪  <expressao> ] ")"

(* ——— FORK E JOIN (2026.2) ——— *)
<expressao_fork> ::= "fork" <chamada> [
↪  "on" <destino_fork> ]
<chamada>        ::= <chamada_funcao> |
↪  <chamada_metodo>
<destino_fork>   ::= "core" "("
↪  <expressao> ")"
                   | "host" "("
                   ↪  <expressao> ","
                   ↪  <expressao> ")"
<expressao_join> ::= "join"
↪  <expressao_unaria>

(* ——— EXPRESSÕES (da menor para a maior
↪  precedência) ——— *)
<expressao>       ::= <expressao_fork> |
↪  <expressao_ou>
<expressao_ou>    ::= <expressao_e> {
↪  "||" <expressao_e> }
<expressao_e>     ::= <expressao_comp> {
↪  "&&" <expressao_comp> }
<expressao_comp>  ::= <expressao_termo> {
↪  ( "==" | "!=" | "<" | ">" | "<=" |
↪  ">=" ) <expressao_termo> }
<expressao_termo> ::= <expressao_fator> {
↪  ( "+" | "-" ) <expressao_fator> }
<expressao_fator> ::= <expressao_unaria> {
↪  ( "*" | "/" | "%" ) <expressao_unaria>
↪  }
<expressao_unaria> ::= ( "-" | "+" | "!" )
↪  <expressao_unaria>
                     | "join" <expressao
                     ↪  _unaria>
                     | <expressao_posfixo>
<expressao_posfixo> ::=
↪  <expressao_primaria>
                        { "."
                        ↪  <IDENTIFIER> [
                        ↪  "(" [ <lista
                        ↪  _argumentos> ]
                        ↪  ")" ] | "["
                        ↪  <expressao>
                        ↪  "]" }
<expressao_primaria> ::= <NUMBER> |
↪  <STRING_LITERAL> | <BOOL_LITERAL> |
↪  "null"
```

```
                        | <IDENTIFIER> [
                        ↪  "(" [ <lista_
                        ↪  argumentos> ]
                        ↪  ")" ]
                        | "this" |
                        ↪  <expressao_new>
                        ↪  | <expressao_
                        ↪  c_channel>
                        | <expressao_rece
                        ↪  ive> |
                        ↪  <expressao_inp
                        ↪  ut>
                        | <expressao_list
                        ↪  a> |
                        ↪  <expressao_mat
                        ↪  rix>
                        | "(" <expressao>
                        ↪  ")"
<lista_argumentos> ::= <expressao> { ","
↪  <expressao> }
<chamada_funcao>   ::= <IDENTIFIER> "(" [
↪  <lista_argumentos> ] ")"
```

```
(* ── TIPOS E LITERAIS ── *)
<tipo>           ::= "number" | "int" |
↪  "string" | "bool" | "void" | "list" |
↪  "dict"
<NUMBER>         ::= <digito> { <digito> }
↪  [ "." { <digito> } ]
<STRING_LITERAL> ::= '"' {
↪  <caractere_string> } '"'
<BOOL_LITERAL>   ::= "true" | "false"
<NULL_LITERAL>   ::= "null"
<IDENTIFIER>     ::= ( <letra> | "_" ) {
↪  <letra> | <digito> | "_" }
<caractere_string> ::=
↪  <qualquer_char_exceto_aspas_e_barra> |
↪  "\n" | "\t" | '\"' | "\\"

(* ── COMENTÁRIOS (descartados pelo
↪  léxico) ── *)
<comentario> ::= "#" { <qualquer_char> }
↪  <nova_linha>
```

3 ANALISADOR LÉXICO

O analisador léxico (`src/lexer.py`, classe `Lexer`) lê o código-fonte caractere a caractere e produz uma lista de tokens. Cada `Token` guarda o tipo (`TokenType`), o lexema, a linha e a coluna. As linhas e colunas são usadas depois nas mensagens de erro e na tabela de tokens da interface Web 2.

Categorias de tokens

Quadro 2 – Categorias de tokens do analisador léxico

Categoria	Exemplos	TokenType
Identificadores	`soma`, `MiniCompilador`, `_aux`	`IDENTIFIER`
Literais	`42`, `3.14`, `"texto"`, `true`, `null`	`NUMBER`, `STRING_LITERAL`, `BOOL_LITERAL`, `NULL`
Palavras reservadas	`class`, `func`, `var`, `if`, `while`, `for`, `seq`, `par`, `c_channel`, `send`, `receive`…	uma por palavra
Fork/Join (2026.2)	`fork`, `join`, `on`, `core`, `host`	`FORK`, `JOIN`, `ON`, `CORE`, `HOST`
Tipos	`number`, `int`, `string`, `bool`, `void`, `list`, `dict`	`TYPE`
Operadores	`+ - * / % = == != < > <= >= && || !`	um por operador
Delimitadores	`( ) { } [ ] ; , .`	um por símbolo
Fim e desconhecido	fim do arquivo, `@`	`END_OF_FILE`, `UNKNOWN`

Fonte: Elaborado pelos autores (2026).

As palavras reservadas ficam em um dicionário `KEYWORDS`. O léxico lê a palavra inteira como se fosse um identificador e só depois consulta o dicionário. Com isso `format` continua sendo identificador, enquanto `for` vira `FOR`.

Pseudocódigo

```
função TOKENIZAR(fonte):
    tokens ← lista vazia
    pos ← 0; linha ← 1; coluna ← 1

    enquanto pos < tamanho(fonte):
        PULAR_ESPAÇOS_E_COMENTÁRIOS()
        se pos ≥ tamanho(fonte): pare
        c ← fonte[pos]
        (linhaIni, colIni) ← (linha, coluna)

        se c é letra ou '_':
            palavra ← ler enquanto letra, dígito ou '_'
            tipo ← KEYWORDS[palavra] se existir, senão IDENTIFIER
            adicionar Token(tipo, palavra, linhaIni, colIni)

        senão se c é dígito:
            num ← ler dígitos
            se próximo = '.': num ← num + '.' + ler dígitos
            adicionar Token(NUMBER, num, linhaIni, colIni)

        senão se c = '"':
            avançar                                  // descarta a aspa de abertura
            texto ← ""
            enquanto próximo ∉ {'"', FIM}:
                se próximo = '\\':
                    avançar; e ← avançar
                    texto ← texto + ESCAPE(e)        // \n, \t, \", \\
                senão texto ← texto + avançar
            se próximo = '"': avançar                // aspa de fechamento
            adicionar Token(STRING_LITERAL, texto, linhaIni, colIni)

        senão:                                       // operadores e delimitadores
            op ← avançar
            se op ∈ {'=', '!', '<', '>'} e próximo = '=':
                avançar; adicionar operador de dois caracteres (==, !=, <=, >=)
            senão se op = '&' e próximo = '&': avançar; adicionar AND
            senão se op = '|' e próximo = '|': avançar; adicionar OR
            senão se op está na tabela de símbolos simples: adicionar o token
            ↪  correspondente
            senão adicionar Token(UNKNOWN, op, linhaIni, colIni)

    adicionar Token(END_OF_FILE, "", linha, coluna)
    retornar tokens

função PULAR_ESPAÇOS_E_COMENTÁRIOS():
    enquanto pos < tamanho(fonte):
        se fonte[pos] é espaço, tab ou quebra de linha: avançar
        senão se fonte[pos] = '#':
            enquanto próximo ∉ {'\n', FIM}: avançar   // comentário até o fim da linha
```

```
        senão pare

função AVANÇAR():
    c ← fonte[pos]; pos ← pos + 1
    se c = '\n': linha ← linha + 1; coluna ← 1
    senão coluna ← coluna + 1
    retornar c
```

Os comentários # são descartados aqui e não chegam ao analisador sintático. Um caractere desconhecido não interrompe a análise léxica: vira um token `UNKNOWN`, e o analisador sintático acusa o erro com a linha correta.

4 ANALISADOR SINTÁTICO

O analisador sintático (`src/parser.py`, classe `Parser`) é **descendente recursivo**: há uma função para cada não terminal da gramática, e cada função consome os tokens da sua produção e devolve um nó da AST (`src/ast_nodes.py`). Ele olha apenas o próximo token para escolher a produção (LL(1)). A única exceção é `int(`, em que o token seguinte também é consultado para distinguir a função de conversão `int(x)` de uma declaração `int x = …`.

A precedência dos operadores é dada pela própria cadeia de chamadas: cada nível chama o nível de precedência maior e repete enquanto encontra um operador do seu nível. Com isso, as operações binárias ficam associativas à esquerda.

Quadro 3 – Níveis de precedência no analisador sintático

Nível (menor → maior)	Função	Operadores
1	`_parse_expression`	`fork … on …`
2	`_parse_or`	`||`
3	`_parse_and`	`&&`
4	`_parse_comparison`	`== != < > <= >=`
5	`_parse_term`	`+ -`
6	`_parse_factor`	`* / %`
7	`_parse_unary`	`- + !` e `join` (prefixos)
8	`_parse_primary`	literais, identificadores, chamadas, `new`, `(…)`, seguidos de `.campo`, `.metodo(…)` e `[i]`

Fonte: Elaborado pelos autores (2026).

Pseudocódigo: programa e comandos

```
função PARSE():
    prog ← Program()
```

```
    enquanto próximo ≠ END_OF_FILE:
        prog.comandos.adicionar(COMANDO())
    retornar prog

função COMANDO():
    escolha próximo token:
        class    → retornar DECL_CLASSE()
        func     → retornar DECL_FUNÇÃO()
        return   → consumir; v ← (próximo = ';') ? nulo : EXPRESSÃO(); opcional ';';
        ↪  retornar ReturnStmt(v)
        break / continue → consumir; opcional ';'; retornar BreakStmt / ContinueStmt
        var      → retornar DECL_VARIÁVEL("var")
        TYPE (exceto int seguido de '(') → t ← consumir; retornar DECL_VARIÁVEL(t)
        seq ou '{' → retornar BLOCO_SEQ()
        par      → retornar BLOCO_PAR()
        print    → retornar PRINT()
        send     → retornar SEND()
        if / while / do / for → retornar o comando correspondente
        IDENTIFIER, this, new, receive, input, c_channel, matrix, '[', fork, join:
            e ← EXPRESSÃO()
            se casar '=': retornar ATRIBUIÇÃO(alvo = e)    // x = …, obj.campo = …,
            ↪  lista[i] = …
            se e é chamada de função, chamada de método, fork ou join:
                consumir ';'; retornar e                    // comando isolado
            ERRO("Instrução ou expressão isolada inválida")
        outro → ERRO("Instrução não reconhecida")

função DECL_FUNÇÃO():
    nome ← consumir IDENTIFIER
    consumir '('; params ← lista de [tipo] IDENTIFIER separados por ','; consumir ')'
    corpo ← BLOCO_SEQ()
    retornar FuncDecl(nome, params, corpo)

função DECL_CLASSE():
    nome ← consumir IDENTIFIER
    super ← se casar extends: consumir IDENTIFIER senão nulo
    consumir '{'
    enquanto próximo ≠ '}':
        se casar var: atributos.adicionar(DECL_VARIÁVEL("var"))
        senão se casar func: métodos.adicionar(DECL_FUNÇÃO())
        senão ERRO("Esperado 'var' ou 'func' na definição da classe")
    consumir '}'
    retornar ClassDecl(nome, super, atributos, métodos)

função COMANDO_FOR():
    consumir '('
    se próximo = var e o token depois do nome = in:          // for (var x in lista)
        nome ← IDENTIFIER; consumir in; iter ← EXPRESSÃO()
    senão:                                                    // for (ini; cond; inc)
        ini ← opcional; consumir ';'; cond ← opcional; consumir ';'; inc ← opcional
    consumir ')'; corpo ← COMANDO()
    retornar ForStmt(…)
```

Pseudocódigo: expressões

```
função EXPRESSÃO():
    se casar fork: retornar FORK()
```

```
    retornar OU()

função FORK():                        // "fork" <chamada> [ "on" <destino> ]
    chamada ← PRIMÁRIA()
    se chamada não é FuncCallExpr nem MethodCall:
        ERRO("'fork' exige uma chamada de função ou método")
    destino ← nulo
    se casar on:
        se casar core: consumir '('; n ← EXPRESSÃO(); consumir ')'; destino ←
        ↪  CoreTarget(n)
        senão se casar host:
            consumir '('; ip ← EXPRESSÃO(); consumir ','; porta ← EXPRESSÃO(); consumir
            ↪  ')'
            destino ← HostTarget(ip, porta)
        senão ERRO("Esperado 'core(n)' ou 'host(ip, porta)' após 'on'")
    retornar ForkExpr(chamada, destino)

função OU():  e ← E();          enquanto casar '||': e ← Binário('||', e, E());
↪  retornar e
função E():   e ← COMPARAÇÃO(); enquanto casar '&&': e ← Binário('&&', e, COMPARAÇÃO());
↪  retornar e
função COMPARAÇÃO(): e ← TERMO();  enquanto próximo ∈ {==,!=,<,>,<=,>=}: e ← Binário(op,
↪  e, TERMO());  retornar e
função TERMO():      e ← FATOR();  enquanto próximo ∈ {+,-}:   e ← Binário(op, e,
↪  FATOR());  retornar e
função FATOR():      e ← UNÁRIA(); enquanto próximo ∈ {*,/,%}: e ← Binário(op, e,
↪  UNÁRIA()); retornar e

função UNÁRIA():
    se casar join: retornar JoinExpr(UNÁRIA())       // join t1 + join t2 = (join t1) +
    ↪  (join t2)
    se próximo ∈ {-, +, !}: op ← consumir; retornar Unário(op, UNÁRIA())
    retornar PRIMÁRIA()

função PRIMÁRIA():
    escolha próximo token:
        NUMBER / STRING / BOOL / null → literal correspondente
        this → ThisExpr
        new  → nome; '(' argumentos ')' → NewExpr
        IDENTIFIER (ou int seguido de '(') → se casar '(': FuncCallExpr(nome, args) senão
        ↪  IdentifierExpr
        '[' → lista de expressões separadas por ',' → ListLiteral
        matrix / c_channel / receive / input → nó correspondente com seus argumentos
        '(' → e ← EXPRESSÃO(); consumir ')'
        outro → ERRO("Expressão esperada")
    repita:                                          // sufixos
        se casar '.': nome ← IDENTIFIER
            se casar '(': nó ← MethodCall(nó, nome, args) senão nó ← PropertyAccess(nó,
            ↪  nome)
        senão se casar '[': i ← EXPRESSÃO(); consumir ']'; nó ← IndexExpr(nó, i)
        senão pare
    retornar nó

função CONSUMIR(tipo, mensagem):
    se próximo.tipo = tipo: retornar avançar()
    ERRO_SINTÁTICO(linha do próximo token, mensagem + " (Encontrado: lexema)")
```

`fork` fica no nível mais baixo porque cria uma tarefa e não se combina com outros operadores. `join` é unário porque o seu resultado é o valor devolvido pela subtarefa e pode entrar em contas: `join t1 + join t2`. Quando há erro sintático, o compilador mostra a linha e o token encontrado e para, sem gerar código.

5 ANALISADOR SEMÂNTICO E TRATAMENTO DE ERROS

O analisador semântico (`src/semantic.py`, classe `SemanticAnalyzer`) percorre a AST depois que ela está completa. Ele trabalha em duas passadas:

1. **Coleta de declarações** (`_collect_declarations`): registra todas as funções e classes globais com seus parâmetros, atributos e métodos. Isso permite chamar uma função declarada mais abaixo no arquivo e escrever funções mutuamente recursivas, como os métodos do parser do Teste 11.
2. **Validação** (`validate_node`): visita cada nó, abre e fecha escopos, define símbolos e confere as regras abaixo.

Ao contrário do sintático, o semântico não para no primeiro erro: ele relata todos e só no fim devolve `false`, o que aborta a compilação antes da geração de código. Cada erro também é guardado em `diagnostics` (linha, tipo, mensagem), que as interfaces Web usam para marcar a linha no editor.

5.1 Regras verificadas

Quadro 4 – Regras verificadas pelo analisador semântico

Regra	Exemplo de erro
Variável usada sem declaração	`Variavel 'x' nao declarada.`
Variável declarada duas vezes no mesmo escopo	`Variavel 'x' ja declarada.`
Tipo incompatível em declaração ou atribuição	`Tipo incompativel em 'n': esperado number, recebido string.`
Operador aritmético (`- * / %`) com operando não numérico	`Operador '*' exige operandos numericos.`
Função inexistente ou com número errado de argumentos	`Funcao 'soma' espera 2 argumento(s), recebeu 1.`
Função nativa com aridade errada	`Funcao nativa 'write_file' espera 2 argumento(s), recebeu 1.`
Classe, superclasse, atributo ou método inexistente	`Atributo 'valor' nao declarado em 'No'.`
`break`/`continue` fora de laço	`'break' usado fora de laco.`
`fork` em `core(n)` com n não inteiro ou menor que 1	`core(0) invalido: os nucleos sao numerados com inteiros a partir de 1.`

Regra	Exemplo de erro
`fork` em `host(ip, porta)` com tipos ou porta inválidos	`Porta 70000 invalida: use um inteiro entre 1 e 65535.`
`fork` em `host` de método ou de função nativa	`fork para host(...) exige uma funcao global …`
Argumento não serializável enviado para outro computador	`Argumento 1 de 'f' (tipo No) nao pode ser enviado a outro computador …`
`join` de algo que não é tarefa	`'join' espera uma tarefa criada por fork, recebeu number.`
`join` repetido na mesma tarefa	`Tarefa 't' ja foi aguardada com join (join duplicado).`

Fonte: Elaborado pelos autores (2026).

Além dos erros, há dois **avisos**, que não interrompem a compilação: `fork` usado como comando isolado, cujo resultado se perde, e tarefa criada com `fork` que nunca é usada em `join`.

A compatibilidade de tipos (`_types_compatible`) aceita `int` e `number` entre si e trata `var`/`unknown` como compatíveis com qualquer tipo, porque a linguagem tem tipagem dinâmica: `var n = len(lista)` só tem tipo conhecido em execução. `null` é aceito em qualquer variável de objeto, mas não em `number`, `int`, `string` ou `bool`.

5.2 Pseudocódigo

```
função ANALISAR(programa):
    COLETAR_DECLARAÇÕES(programa)                // 1ª passada: funções e classes
    ↪  globais
    para cada comando em programa: VALIDAR_COMANDO(comando)
    AVISAR_TAREFAS_SEM_JOIN()
    retornar não houveErros

função VALIDAR(nó):
    escolha tipo de nó:
        VarDecl:
            VALIDAR(nó.valorInicial)
            real ← INFERIR_TIPO(nó.valorInicial)
            declarado ← (nó.tipo = "var" e real ≠ "unknown") ? real : nó.tipo
            se não COMPATÍVEIS(declarado, real): ERRO("Tipo incompatível")
            DEFINIR(nó.nome, declarado)
            se nó.valorInicial é ForkExpr: registrar tarefa (símbolo, profundidade de
            ↪  laço)
        Assignment:
            s ← BUSCAR(nó.nome); se s = nulo: ERRO("não declarada")
            VALIDAR(nó.valor)
            se nó.valor é ForkExpr: marcar s como ainda não aguardada
            se não COMPATÍVEIS(s.tipo, INFERIR_TIPO(nó.valor)): ERRO
        IdentifierExpr:
            se BUSCAR(nó.nome) = nulo: ERRO("não declarada")
        BinaryExpr:
```

```
            VALIDAR(esq); VALIDAR(dir)
            se op ∈ {-, *, /, %} e algum lado não é numérico: ERRO
        SeqBlock, ParBlock:
            ABRIR_ESCOPO(); validar cada comando; FECHAR_ESCOPO()
        IfStmt:
            VALIDAR(condição)
            antes ← aguardadas
            validar ramo então; depoisEntão ← aguardadas
            aguardadas ← antes; validar ramo senão
            aguardadas ← depoisEntão ∩ aguardadas  // só conta join feito nos dois ramos
        WhileStmt, DoWhileStmt, ForStmt:
            (for abre escopo e define a variável do laço)
            profundidadeLaço++; VALIDAR(corpo); profundidadeLaço--
        BreakStmt, ContinueStmt:
            se profundidadeLaço = 0: ERRO("fora de laço")
        FuncDecl:
            DEFINIR(nome, "FUNC", params); ABRIR_ESCOPO()
            para cada p em params: DEFINIR(p, "var")
            VALIDAR(corpo); FECHAR_ESCOPO()
        ClassDecl:
            se superclasse não existe: ERRO
            DEFINIR(nome, "CLASS"); ABRIR_ESCOPO(); classeAtual ← nome
            validar atributos e métodos; FECHAR_ESCOPO()
        FuncCallExpr:
            se nome não é função do programa nem nativa: ERRO
            se quantidade de argumentos difere: ERRO
        MethodCall, PropertyAccess, PropertyAssign:
            classe ← INFERIR_TIPO(objeto)
            se a classe é conhecida e não tem o método/atributo: ERRO
        NewExpr:
            se classe não existe: ERRO
        ForkExpr: VALIDAR_FORK(nó)
        JoinExpr: VALIDAR_JOIN(nó)

função VALIDAR_FORK(nó):
    VALIDAR(nó.chamada)                          // função existe e aridade confere
    se destino é core(n):
        se n não é numérico, ou é literal fracionário ou < 1: ERRO
    se destino é host(ip, porta):
        se ip não é texto ou porta não é numérica: ERRO
        se porta é literal fora de 1..65535: ERRO
        se chamada é método ou função nativa: ERRO
        para cada argumento: se tipo ∉ {number, int, string, bool, list, matrix}: ERRO

função VALIDAR_JOIN(nó):
    VALIDAR(nó.tarefa)
    se INFERIR_TIPO(nó.tarefa) ∉ {task, var, unknown}: ERRO("join espera uma tarefa")
    se nó.tarefa é variável s, criada nesta mesma profundidade de laço:
        se s ∈ aguardadas: ERRO("join duplicado")
        senão aguardadas ← aguardadas ∪ {s}
```

A verificação de join duplicado considera a profundidade de laço. Dentro de um `for`, a linha `var t = fork f(i); … join t;` roda várias vezes com tarefas diferentes, e isso não é erro.

5.3 Tratamento de erros em cada fase

Quadro 5 – Tratamento de erros em cada fase do compilador

Fase	Como o erro aparece	O que acontece
Léxica	Caractere desconhecido vira token `UNKNOWN`	O sintático acusa o erro na linha do caractere
Sintática	`[ERRO SINTATICO] Linha 7: Esperado ';' (Encontrado: 'print')`	Para no primeiro erro, sem gerar código
Semântica	`[ERRO SEMANTICO] Linha 12 (Funcao: calcular): Variavel 'y' nao declarada.`	Lista todos os erros e aborta a compilação
Compilação C++	Mensagem do `g++`	O pipeline informa a falha da etapa final
Execução	`[ERRO DE EXECUCAO] Divisao por zero.` , índice fora dos limites, worker remoto inacessível	O programa termina com código de erro e mensagem clara

Fonte: Elaborado pelos autores (2026).

Os comentários (`# …`) não precisam de tratamento semântico: o léxico já os descarta. Os erros de execução do `fork` remoto incluem a falta de conexão com o worker (`nao foi possivel conectar a ip:porta`) e o worker sem a função pedida (`compile o mesmo programa nos dois computadores`).

6 TABELA DE SÍMBOLOS

A tabela de símbolos fica dentro do analisador semântico e é uma **pilha de escopos**. Cada escopo é um dicionário `nome → Symbol`. O escopo global é criado no construtor. Blocos `seq`/`par`, laços `for`, funções e classes empilham um escopo novo ao entrar e o desempilham ao sair. A busca começa no escopo do topo e desce até o global, e assim uma variável local esconde uma global de mesmo nome.

Além da pilha, há duas tabelas globais preenchidas na primeira passada: `_functions` (nome → parâmetros) e `_classes` (nome → atributos com tipos e métodos com parâmetros).

Estrutura de um símbolo

Quadro 6 – Campos de um símbolo

Campo	Conteúdo
`name`	Nome do identificador

Campo	Conteúdo
`type`	`number`, `int`, `string`, `bool`, `list`, `matrix`, `c_channel`, `task`, `null`, nome de classe, `FUNC`, `CLASS` ou `var` (desconhecido)
`params`	Parâmetros, para funções e métodos
`attributes`	Atributo → tipo, para classes
`methods`	Método → parâmetros, para classes
`loop_depth`	Profundidade de laço em que uma tarefa (`fork`) foi criada, usada na verificação de join duplicado

Fonte: Elaborado pelos autores (2026).

Pseudocódigo

```
estrutura Símbolo: nome, tipo, params, atributos, métodos, profundidadeLaço

escopos ← [ {} ]                          // pilha; posição 0 = escopo global

função ABRIR_ESCOPO():   empilhar {} em escopos
função FECHAR_ESCOPO():  se tamanho(escopos) > 1: desempilhar   // nunca remove o global

função DEFINIR(nome, tipo, linha, params = []):
    se nome ∈ topo(escopos):
        ERRO(linha, "Variável 'nome' já declarada.")
    senão:
        topo(escopos)[nome] ← Símbolo(nome, tipo, params)

função BUSCAR(nome):
    para escopo em escopos, do topo para a base:
        se nome ∈ escopo: retornar escopo[nome]
    retornar nulo

função INFERIR_TIPO(expressão):
    escolha expressão:
        número → "number";  texto, input, receive → "string";  true/false → "bool"
        null → "null";  […] → "list";  matrix → "matrix";  c_channel → "c_channel"
        fork → "task";  join → "unknown"            // depende do retorno da subtarefa
        new C(…) → "C";  this → classe atual
        identificador → BUSCAR(nome).tipo (ou "unknown")
        a + b com algum lado texto → "string";  com algum lado lista → "list"
        comparação ou lógico → "bool";  outra aritmética → "number"
        obj.campo → tipo do atributo na classe de obj
        funções nativas → tipo conhecido (len → unknown, range → list, int/floor → int…)
        outro → "unknown"
```

Exemplo de tabela durante a análise do Teste 8 (paralelo com host)

Quadro 7 – Tabela de símbolos durante a análise do Teste 8 com host

Escopo	Nome	Tipo	Observação
global (`_functions`)	`avaliar_lote`	FUNC	params: `inicio`, `fim`
global	`ip`	string	lido com `input`
bloco `seq`	`inicio_seq`, `tempo_seq`, `inicio_par`, `tempo_par`	number	calculados com `clock()`
bloco `seq`	`seq1` a `seq4`	var	retorno de função: tipo só conhecido em execução
bloco `seq`	`t1`, `t2`, `t3`	task	criadas com `fork … on core(n)`, profundidade de laço 0
bloco `seq`	`t4`	task	criada com `fork … on host(ip, 5001)`
bloco `seq`	`par1` a `par4`	var	resultado de `join`
bloco `seq`	`soma_seq`, `soma_par`	number	soma com `+`
função `avaliar_lote`	`inicio`, `fim`	var	parâmetros
corpo de `avaliar_lote`	`total`, `tarefa`	number	locais
corpo do `while` externo	`estado`, `iteracoes`, `valor`, `i`	number	locais de cada iteração

Fonte: Elaborado pelos autores (2026).

Na interface Web 2, a tabela de símbolos aparece por escopo para o programa digitado pelo usuário, com as mesmas colunas.

7 INTERFACES DO COMPILADOR

O compilador é iniciado por `python main.py`, que pergunta o modo de uso. Ele também pode ser chamado direto com um arquivo: `python main.py tests/teste8_paralelo.minipar`.

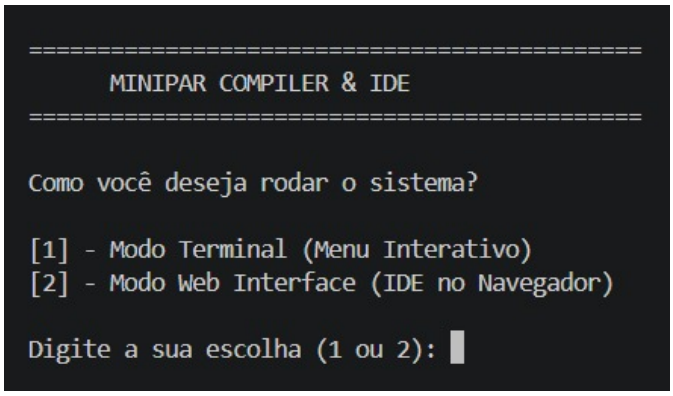
```
==========================================
      MINIPAR COMPILER & IDE
==========================================
[1] - Modo Terminal (Menu Interativo)
[2] - Modo Web Interface (IDE no Navegador)
```

7.1 Terminal

O menu interativo (`menu.py`) lista os programas de `tests/`, mostra uma descrição de cada teste antes de rodar e executa um teste escolhido ou todos em sequência. Para cada programa, ele roda as quatro fases (léxica e sintática, semântica, TAC e C++ com `g++ -O3`) e depois executa o binário gerado em `output/`. Programas que usam `input()` rodam com o teclado conectado.

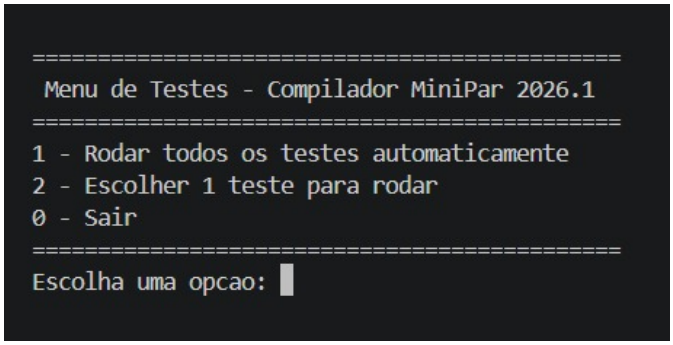
Nos testes com dois terminais ou dois computadores (canais e `fork … on host`), o menu cria uma **sessão de teste** (`test_orchestrator.py`). Um segundo terminal aberto no mesmo projeto detecta a sessão ativa, entra como participante e acompanha o progresso e a saída do primeiro.

Figura 1 – Tela inicial de `python main.py`, com a escolha entre o modo terminal e a interface Web



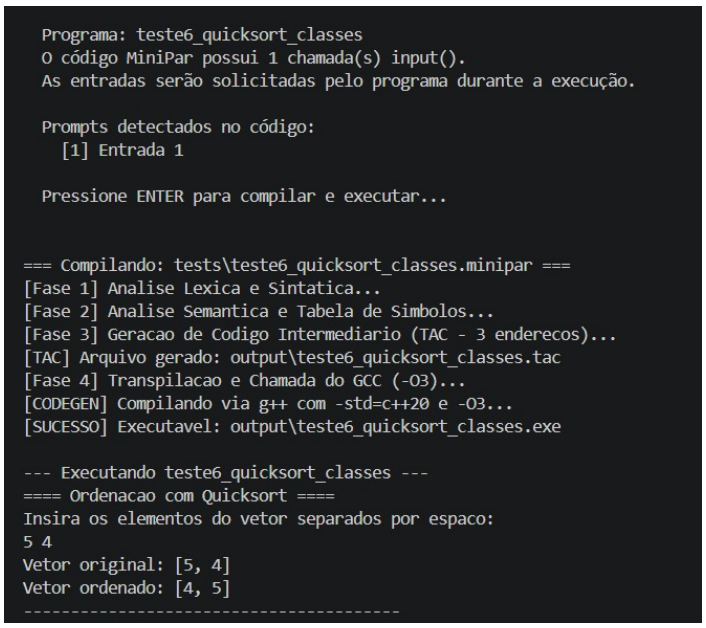
Fonte: Captura de tela elaborada pelos autores (2026).

Figura 2 – Menu do modo terminal: rodar todos os testes ou escolher um



Fonte: Captura de tela elaborada pelos autores (2026).

Figura 3 – Teste 6 escolhido no menu: descrição das entradas, as quatro fases da compilação e a execução com a entrada `5 4`

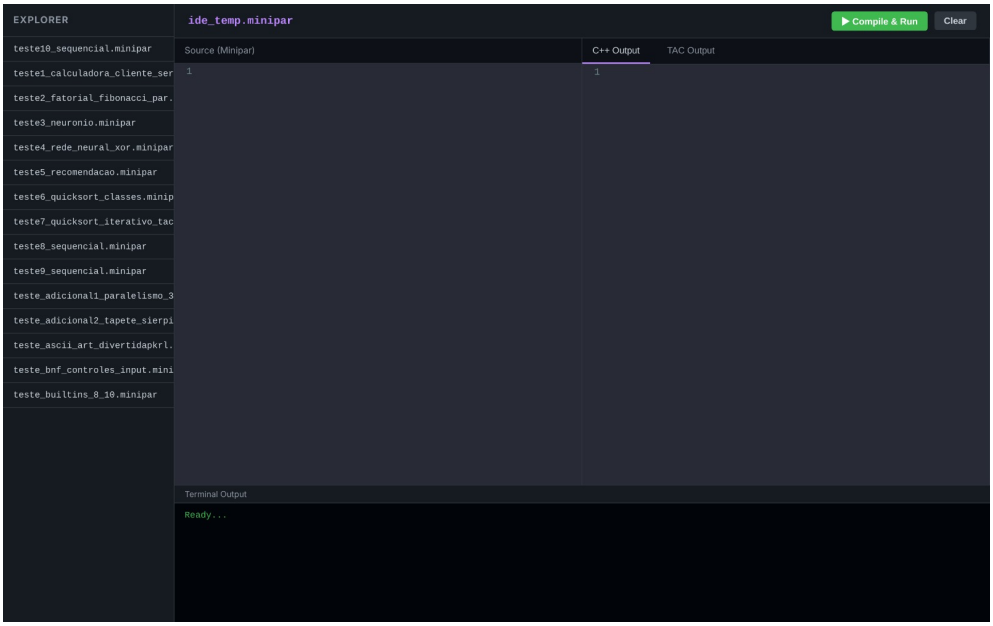


Fonte: Captura de tela elaborada pelos autores (2026).

7.2 Web 1: interface do desenvolvedor

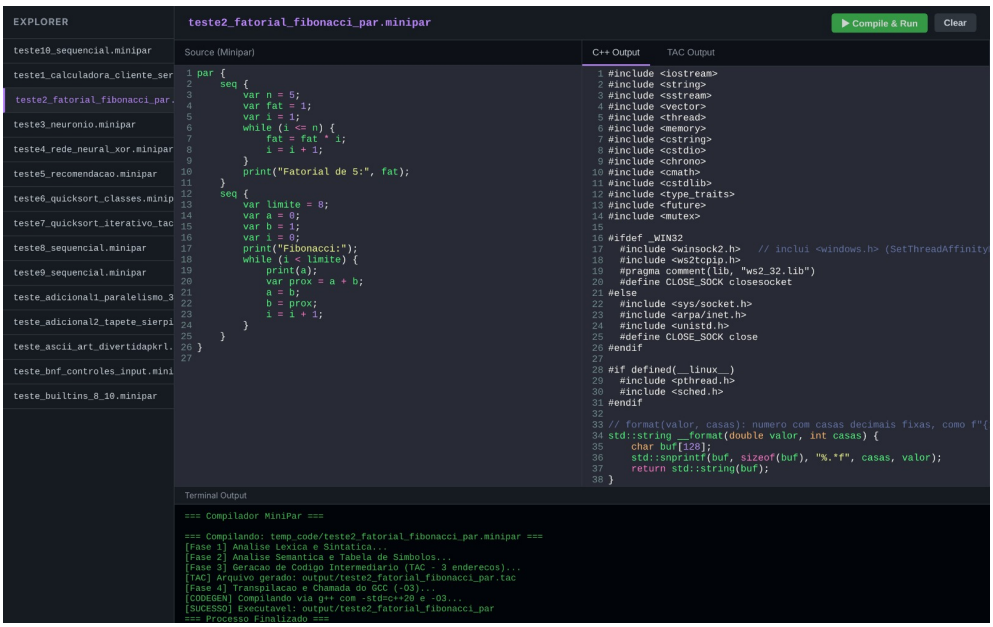
Acessível em `http://127.0.0.1:5000/`. É uma IDE simples: um explorador de arquivos com os programas de teste, um editor de código, o botão **Compile & Run** e abas com o C++ gerado (**C++ Output**) e o código de três endereços (**TAC Output**), além da saída da execução. Ela foi mantida do semestre anterior e serve para compilar e rodar rapidamente.

Figura 4 – Web 1 ao abrir: explorador com os programas de teste, editor vazio e terminal pronto



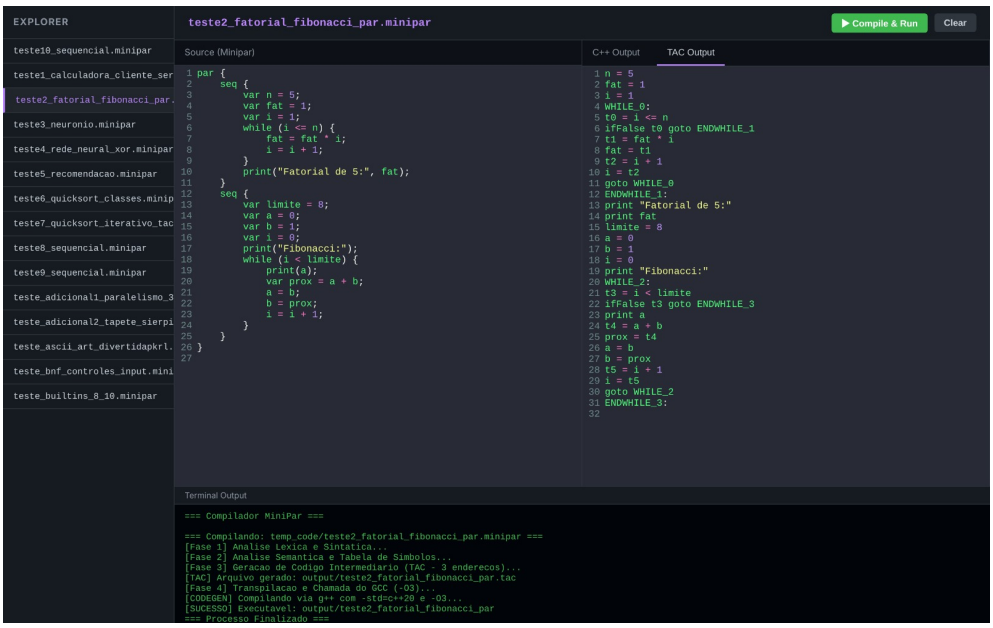
Fonte: Captura de tela elaborada pelos autores (2026).

Figura 5 – Web 1 depois de Compile & Run no Teste 2: código-fonte, C++ gerado e as fases no terminal



Fonte: Captura de tela elaborada pelos autores (2026).

Figura 6 – Web 1, aba TAC Output: código de três endereços do Teste 2, com os laços traduzidos em rótulos e desvios



Fonte: Captura de tela elaborada pelos autores (2026).

7.3 Web 2: interface didática (nova em 2026.2)

Acessível em `http://127.0.0.1:5000/didatico`. O objetivo é mostrar ao aluno o que cada fase do compilador produz para o programa que ele escreveu. O editor usa CodeMirror; o botão **Analisar** (ou Ctrl+Enter) envia o código para o back-end Flask, que roda as fases e devolve o resultado de cada uma em JSON.

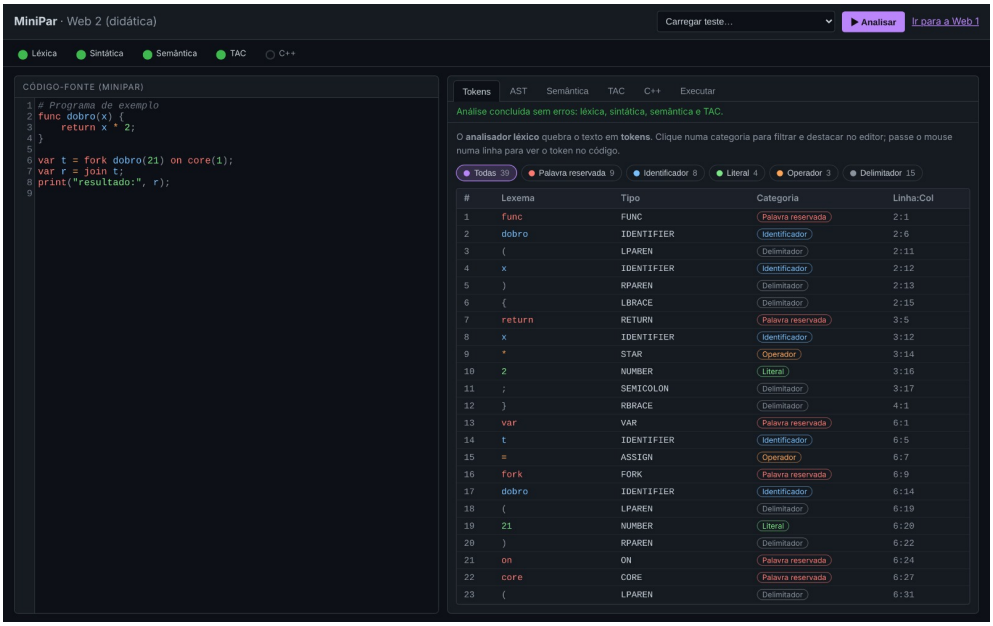
Quadro 8 – Abas da interface Web 2

Aba	O que mostra	Endpoint
Tokens	Tabela com tipo, lexema, linha e coluna, colorida por categoria (palavra reservada, identificador, literal, operador, delimitador); ao passar o mouse, o token é destacado no editor	`/api/tokens`
AST	Árvore gráfica (D3) com nós recolhíveis e botões Expandir tudo, Recolher em comandos e Centralizar	`/api/ast`
Semântica	Erros e avisos em cartões ligados à linha do editor e a tabela de símbolos por escopo	`/api/semantic`
TAC	Código de três endereços com destaque de sintaxe	`/api/tac`
C++	C++20 gerado, com destaque de sintaxe	`/api/cpp`
Executar	Campo de entrada (stdin), botão **Compilar e executar** e **Baixar executável**	`/api/run`, `/api/build`

Fonte: Elaborado pelos autores (2026).

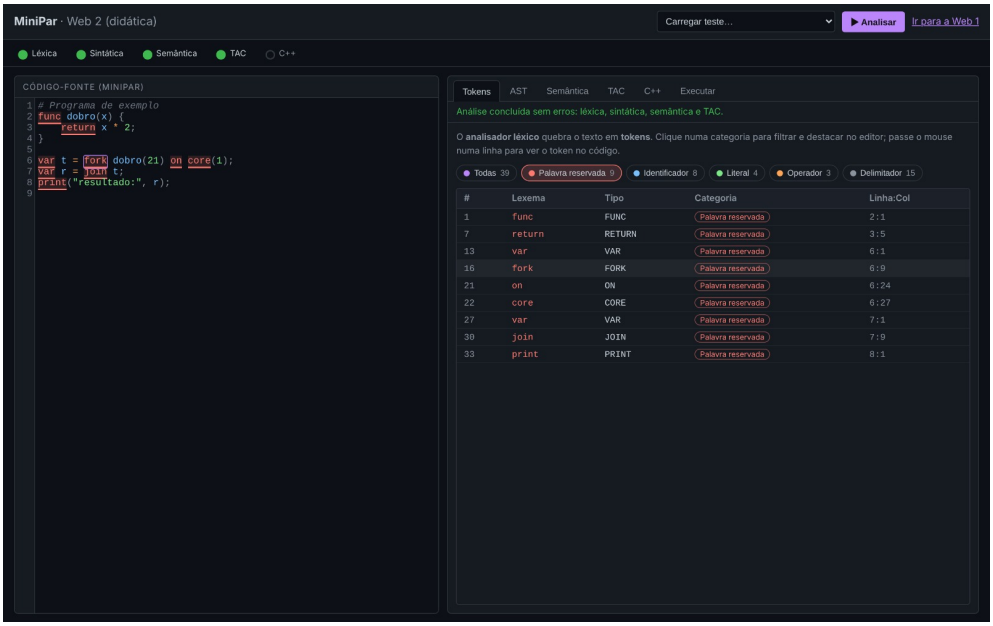
A AST é convertida para JSON por reflexão (`ast_to_dict`): qualquer nó novo, como `ForkExpr` ou `NullExpr`, aparece na árvore sem código extra na interface.

Figura 7 – Aba Tokens: cada token com tipo, categoria e posição (linha:coluna), para um programa com `fork … on core(1)` e `join`



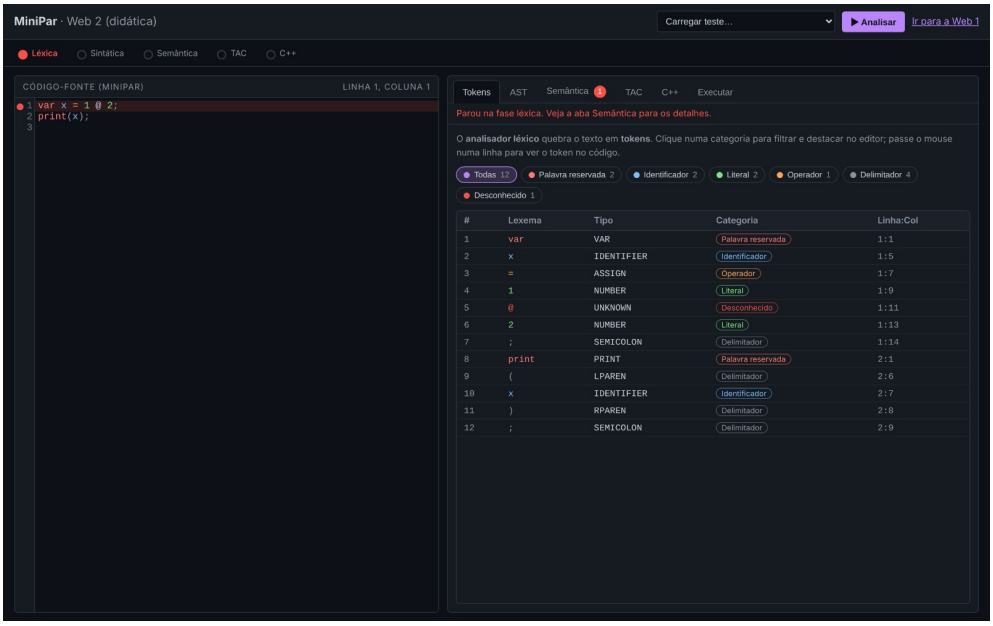
Fonte: Captura de tela elaborada pelos autores (2026).

Figura 8 – Filtro por categoria: só as palavras reservadas, destacadas no editor (incluindo `fork`, `on`, `core` e `join`)



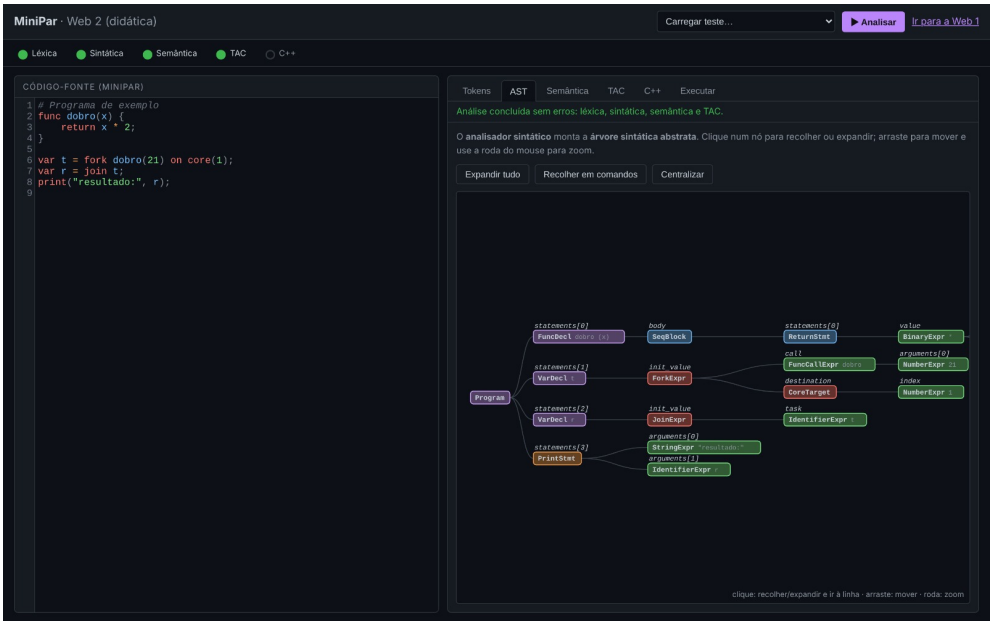
Fonte: Captura de tela elaborada pelos autores (2026).

Figura 9 – Erro léxico: o caractere `@` vira um token `UNKNOWN`, a linha é marcada no editor e a análise para na fase léxica



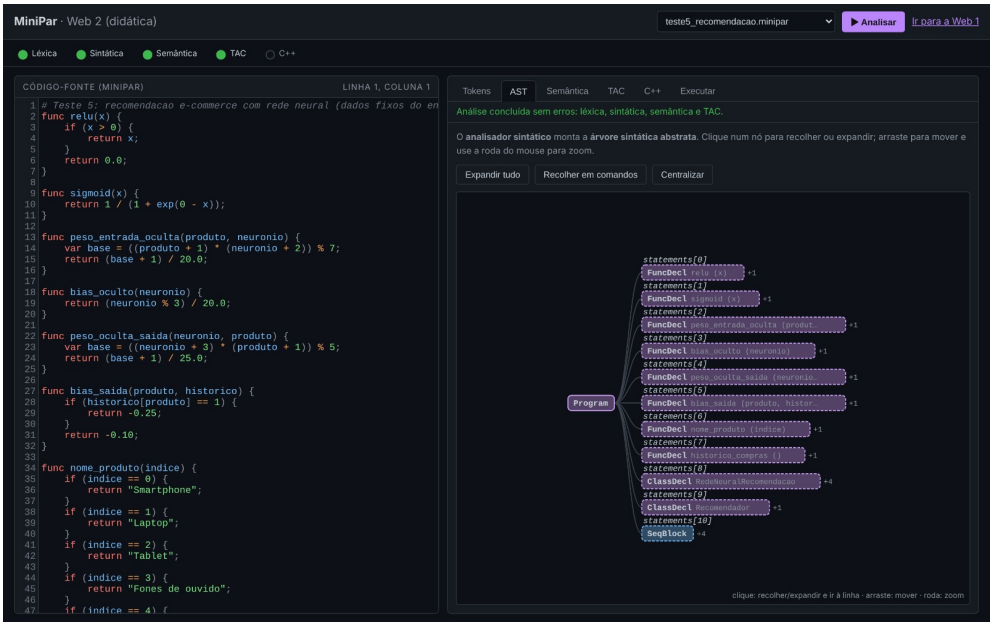
Fonte: Captura de tela elaborada pelos autores (2026).

Figura 10 – Aba AST: os nós `ForkExpr` (com `FuncCallExpr` e `CoreTarget`) e `JoinExpr` do mesmo programa



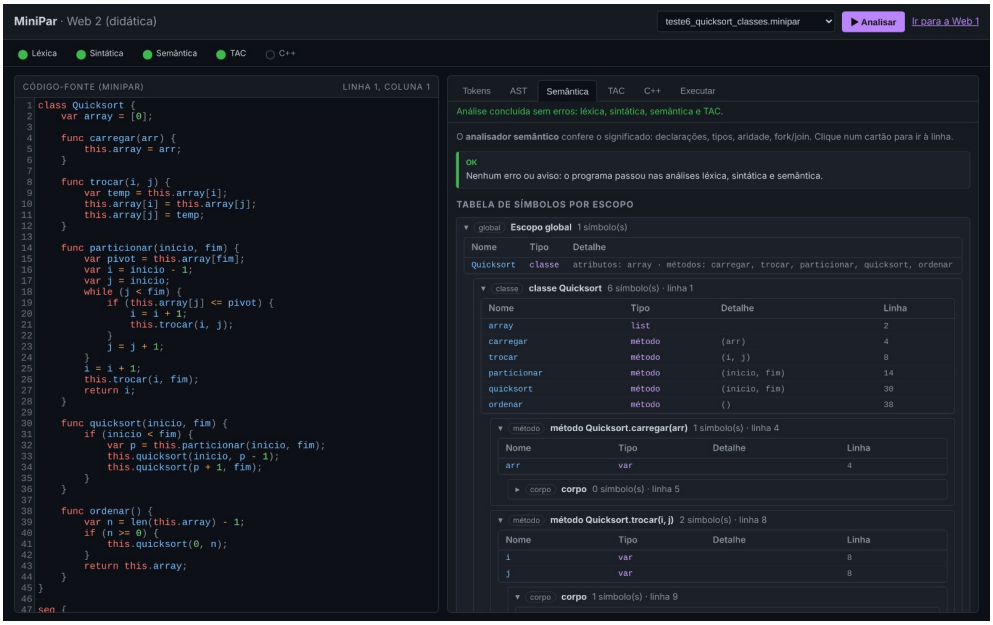
Fonte: Captura de tela elaborada pelos autores (2026).

Figura 11 – AST do Teste 5 recolhida em comandos: funções, classes e o bloco principal



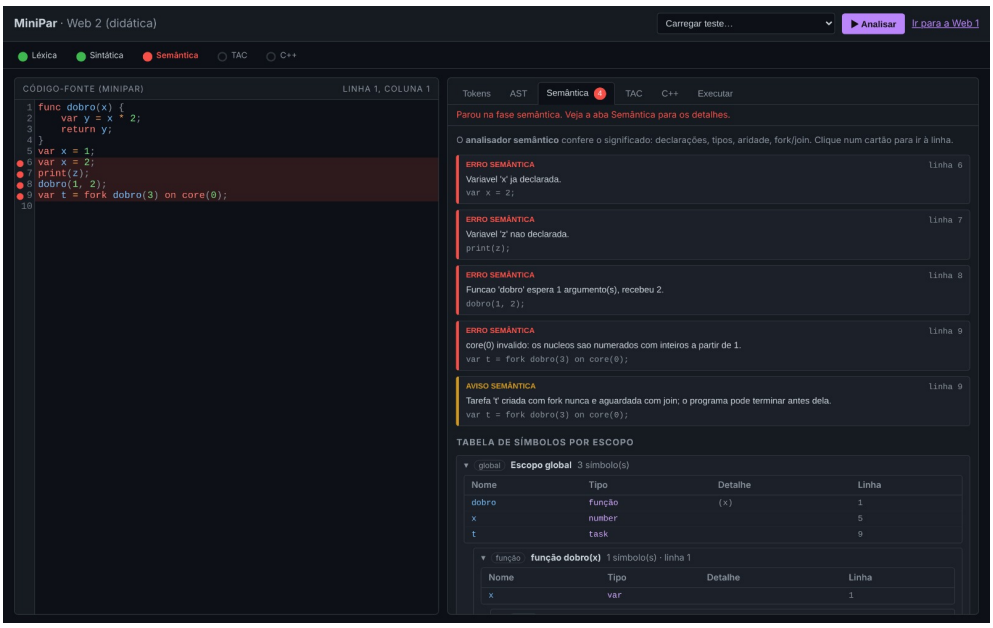
Fonte: Captura de tela elaborada pelos autores (2026).

Figura 12 – Aba Semântica do Teste 6: sem erros e com a tabela de símbolos por escopo (global, classe, método e corpo)



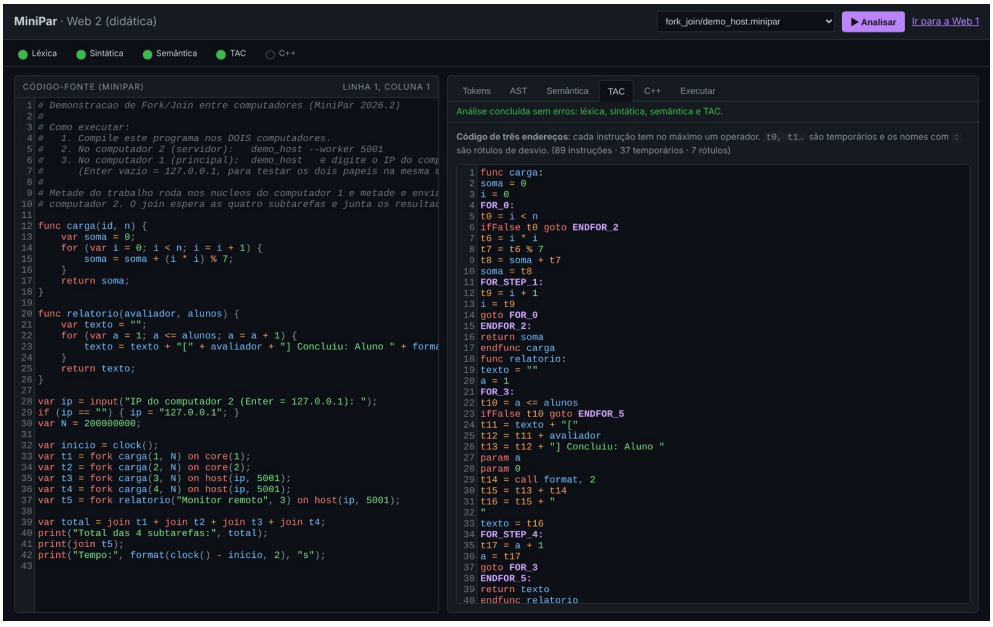
Fonte: Captura de tela elaborada pelos autores (2026).

Figura 13 – Erros semânticos em cartões ligados à linha: variável já declarada, variável não declarada, aridade errada e `core(0)` inválido, além do aviso de tarefa sem `join`



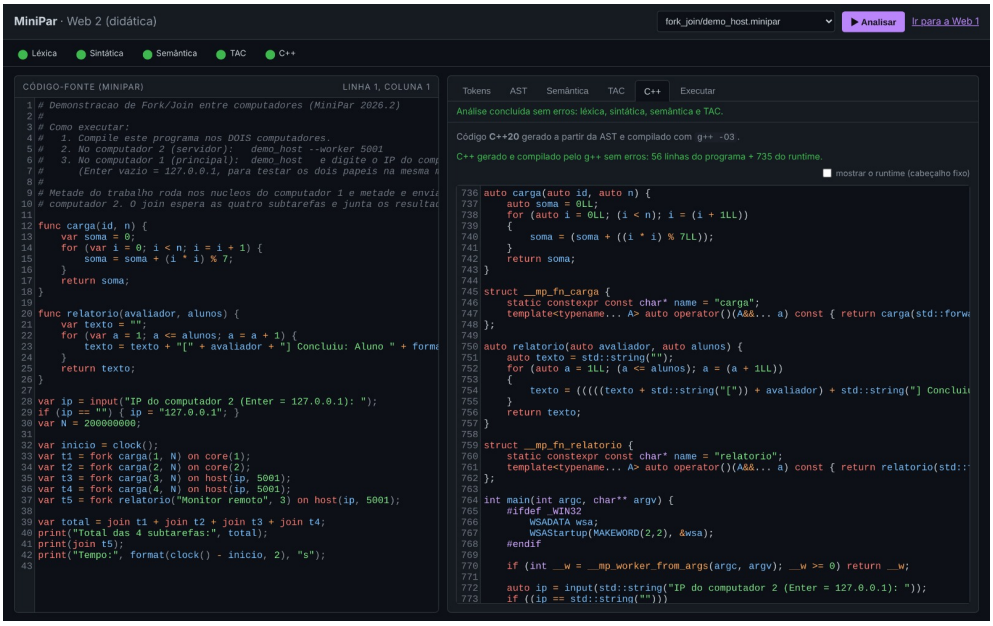
Fonte: Captura de tela elaborada pelos autores (2026).

Figura 14 – Aba TAC do programa `demo_host` (fork em núcleos e em outro computador), com contagem de instruções, temporários e rótulos



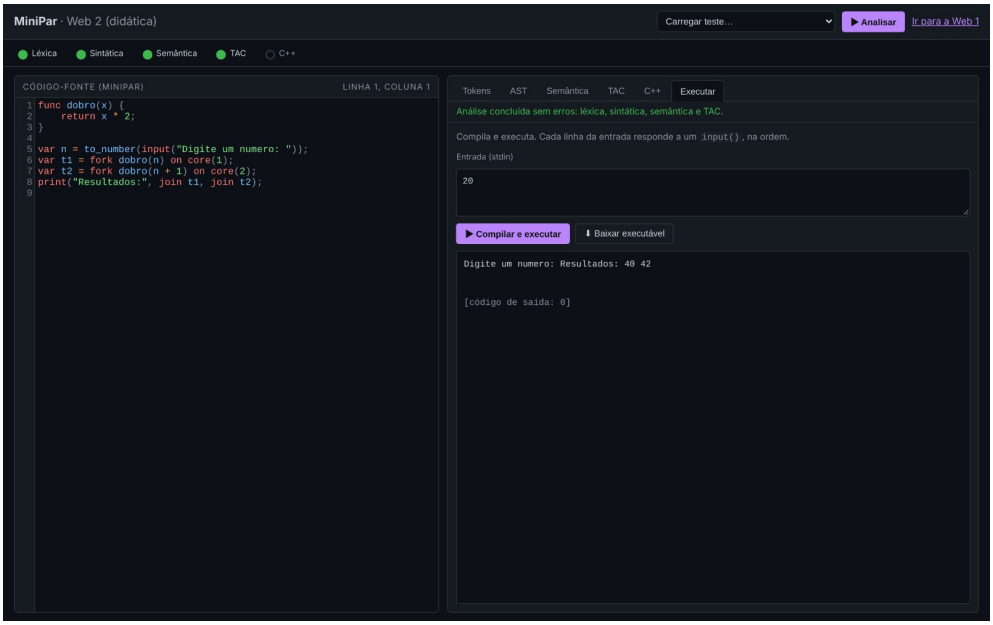
Fonte: Captura de tela elaborada pelos autores (2026).

Figura 15 – Aba C++ do `demo_host`: as funções geradas, os functors `__mp_fn_carga` e `__mp_fn_relatorio` usados no worker e o `main` com `__mp_worker_from_args`



Fonte: Captura de tela elaborada pelos autores (2026).

Figura 16 – Aba Executar: o programa lê 20 pelo campo stdin, roda dois `fork … on core` e imprime os resultados dos `join` (40 e 42)



Fonte: Captura de tela elaborada pelos autores (2026).

8 PRODUCT BACKLOG

O backlog parte do compilador MiniPar 2026.1, que já tinha léxico, sintático, semântico, TAC, geração de C++, canais e a interface Web 1. Os itens abaixo são o que a versão 2026.2 acrescenta ou corrige. Prioridade: **A** = requisito obrigatório do enunciado, **M** = necessário para algum teste, **B** = melhoria.

Quadro 9 – Product backlog da versão 2026.2

ID	História de usuário	Prioridade	Responsável	Situação
PB01	Como programador, quero escrever `fork f(x) on core(n)` e `join t` para dividir o trabalho entre os núcleos da CPU	A	Anthony (front end), Pedro (C++)	Concluído
PB02	Como programador, quero `fork f(x) on host(ip, porta)` para executar uma subtarefa em outro computador	A	Anthony	Concluído
PB03	Como programador, quero erros claros quando uso fork/join de forma errada (join duplicado, núcleo inválido, argumento que não pode ir pela rede)	A	Anthony	Concluído
PB04	Como aluno, quero uma interface didática que mostre tokens, AST, tabela de símbolos, TAC e C++ do meu programa	A	Lucas	Concluído
PB05	Como professor, quero os Testes 8, 9 e 10 em MiniPar, nas versões sequencial e paralela, com medição de tempo	A	Vinicius	Concluído
PB06	Como professor, quero o Teste 11 (minicompilador de expressões para ARMv7) escrito em MiniPar	A	Anthony	Concluído
PB07	Como programador, quero `null`, concatenação de texto com número e `write_file` para escrever programas maiores como o Teste 11	M	Anthony	Concluído
PB08	Como programador, quero `clock()`, `int()`, `floor()`, indexação de string e inteiros de 64 bits para os testes de desempenho	M	Pedro	Concluído
PB09	Como desenvolvedor do grupo, quero uma suíte de regressão (pytest) que compare a saída de todos os testes com o resultado esperado	A	Pedro	Concluído
PB10	Como usuário de Linux e de Windows, quero que o compilador gere e ligue o executável nos dois sistemas	M	Pedro	Concluído
PB11	Como programador, quero que os canais (`send`/`receive`) não travem quando cliente e servidor iniciam em ordens diferentes	M	Anthony	Concluído
PB12	Como programador, quero que `/` seja divisão real e que divisão por zero gere erro em execução	M	Pedro	Concluído
PB13	Como avaliador, quero a gramática BNF 2026.2, os pseudocódigos, a modelagem UML e o relatório	A	Todos	Concluído

ID	História de usuário	Prioridade	Responsável	Situação
PB14	Como programador, quero que `core(n)` aproveite toda a CPU em processadores híbridos (núcleos P e E)	B	—	Não feito (ver 15.c)

Fonte: Elaborado pelos autores (2026).

9 SPRINTS BACKLOG

O trabalho foi feito em três sprints curtas, acompanhadas por um checklist compartilhado com prazo por tarefa e pontos de sincronização entre os quatro integrantes. Antes da Sprint 1, o grupo fixou a sintaxe de fork/join e os nós da AST, para que cada um pudesse trabalhar sem esperar o outro.

9.1 Sprint 1: base e front end

Meta: sintaxe de fork/join reconhecida de ponta a ponta até o TAC; base estável em Linux e Windows; regressão automatizada.

Quadro 10 – Tarefas da Sprint 1

Tarefa	Responsável	Situação
Produções BNF de `fork … on core/host` e `join`; nós `ForkExpr`, `JoinExpr`, `CoreTarget`, `HostTarget`	Anthony	Concluída
Léxico, sintático, semântico e TAC de fork/join	Anthony	Concluída
Ligação multiplataforma (`-lws2_32` e `.exe` só no Windows)	Pedro	Concluída
Correção do Teste 7 e das saídas dos Testes 3, 4 e 5	Pedro	Concluída
Script de regressão com pytest	Pedro	Concluída
Builtins `clock`, `int`, `floor`, indexação de string, inteiro de 64 bits	Pedro	Concluída
Rota `/didatico` e endpoints `/api/tokens`, `/ast`, `/semantic`, `/tac`, `/cpp`, `/build`, `/run`	Lucas	Concluída
Testes 8, 9 e 10 sequenciais e resultados esperados	Vinicius	Concluída

Fonte: Elaborado pelos autores (2026).

Sincronizações: sintaxe e AST definidas; base estável e regressão no repositório.

9.2 Sprint 2: paralelismo, interface e Teste 11

Meta: `fork` executando em núcleos e em outro computador; Web 2 completa; Testes 8 a 11 rodando; código congelado.

Quadro 11 – Tarefas da Sprint 2

Tarefa	Responsável	Situação
C++ de `fork … on core(n)` com `std::thread`, `promise/future` e afinidade de núcleo	Pedro	Concluída
Modo `--worker <porta>`, serialização de argumentos e retorno ao `join`	Anthony	Concluída
Teste de `fork on host` em localhost e entre dois computadores	Anthony e Pedro	Concluída
Correção do travamento dos canais no Teste 1	Anthony	Concluída
`null`, concatenação, `write_file` e tipos de retorno de objetos	Anthony	Concluída
Teste 11 em MiniPar e validação dos três `.arm` no CPULATOR	Anthony	Concluída
Teste 8 com um lote enviado a outro computador	Anthony	Concluída
Tokens por categoria, AST gráfica, escopos, erros em cartões, TAC e C++ com destaque	Lucas	Concluída
Versões paralelas e benchmark dos Testes 8 a 10; Sierpinski do Teste 10	Vinicius	Concluída

Fonte: Elaborado pelos autores (2026).

Sincronizações: `fork on core` funcionando de ponta a ponta; congelamento do código com regressão verde.

9.3 Sprint 3: relatório e apresentação

Meta: relatório completo, teste final em dois computadores e ensaio da apresentação.

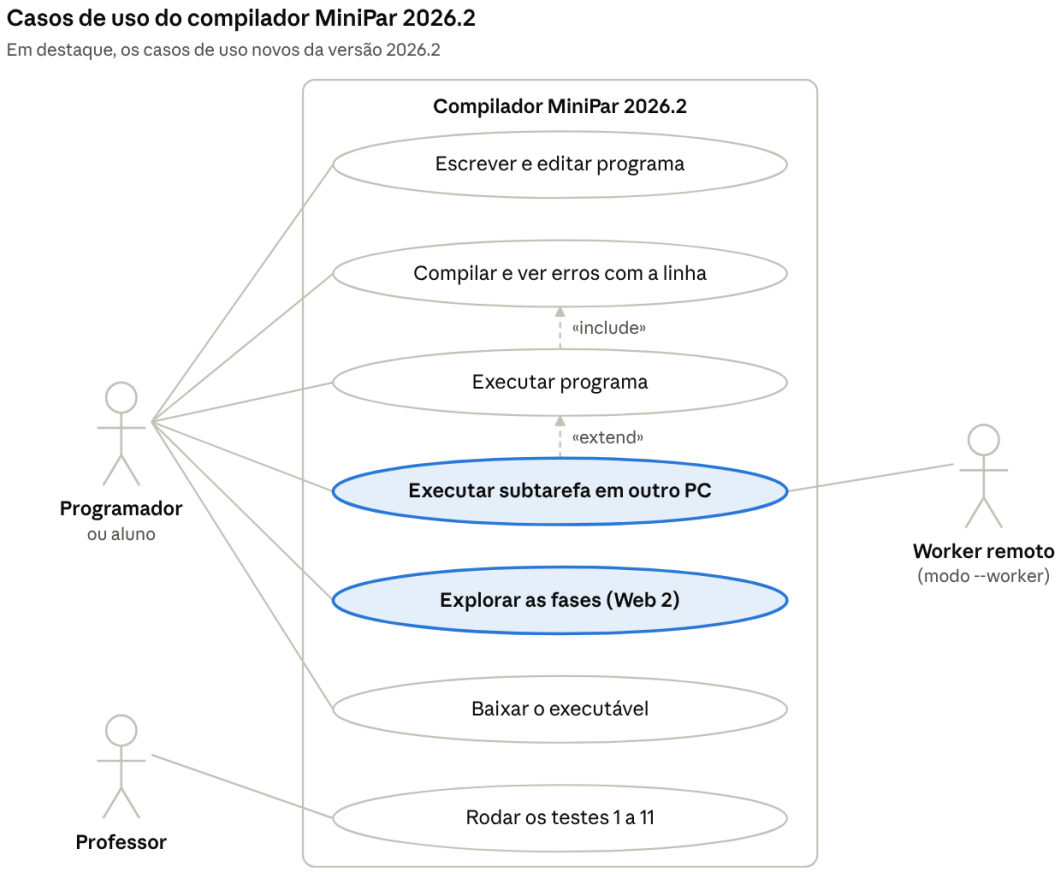
Quadro 12 – Tarefas da Sprint 3

Tarefa	Responsável
BNF 2026.2 e pseudocódigos (seções 2 a 6)	Anthony
Tabela de símbolos, geração de C++, testes de integração e tecnologias	Pedro
Interfaces e prints (seção 7)	Lucas
UML, backlog, sprints, testes de exemplo e consolidação	Vinicius
Ensaio da apresentação	Todos

Fonte: Elaborado pelos autores (2026).

10 DIAGRAMA DE CASOS DE USO

Figura 17 – Diagrama de casos de uso do compilador MiniPar 2026.2



Fonte: Elaborado pelos autores (2026).

Quadro 13 – Descrição dos casos de uso

Caso de uso	Ator	Descrição
Escrever e editar programa	Programador	Escreve o código MiniPar no editor da Web 1, da Web 2 ou em um arquivo `.minipar`
Compilar e ver erros com a linha	Programador	Roda as fases léxica, sintática, semântica, TAC e C++; os erros aparecem com a linha e, na Web 2, em cartões ligados ao editor
Executar programa	Programador	Inclui a compilação («include») e executa o binário gerado, com entrada pelo teclado ou pelo campo stdin

Caso de uso	Ator	Descrição
Executar subtarefa em outro PC	Programador, Worker remoto	Estende a execução («extend») quando o programa usa `fork … on host(ip, porta)`: a chamada vai pela rede para o worker, que executa a função e devolve o resultado ao `join`
Explorar as fases (Web 2)	Programador ou aluno	Vê tokens, AST, tabela de símbolos, TAC e C++ do próprio programa
Baixar o executável	Programador	Baixa o binário gerado pela Web 2 para rodar em outro computador
Rodar os testes 1 a 11	Professor	Escolhe um teste no menu do terminal (ou roda todos) e confere a saída; o pytest automatiza a comparação com as saídas esperadas

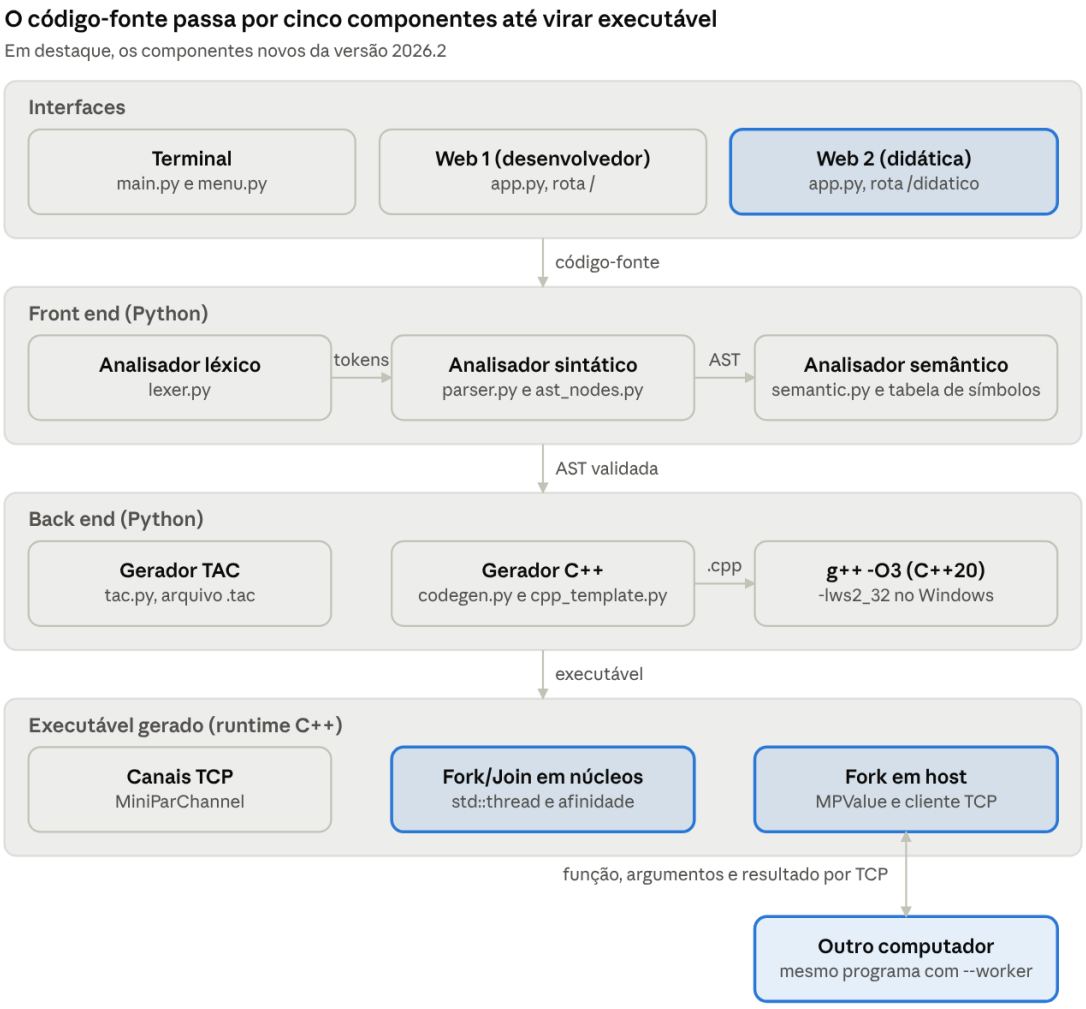
Fonte: Elaborado pelos autores (2026).

O worker remoto é o mesmo programa compilado, iniciado em outro computador com `--worker <porta>`. Ele não escreve código: só recebe chamadas de função e devolve os resultados.

11 ARQUITETURA COM COMPONENTES DE SOFTWARE

O compilador é uma sequência de componentes com interfaces bem definidas: cada um recebe a saída do anterior e não conhece os detalhes internos dos outros. A AST (`ast_nodes.py`) é o contrato central: léxico e sintático a produzem, e o semântico, os geradores e a Web 2 a consomem.

Figura 18 – Arquitetura de componentes do compilador MiniPar 2026.2



Fonte: Elaborado pelos autores (2026).

Quadro 14 – Interfaces dos componentes de software

Componente	Interface fornecida	Interface requerida
Interfaces (terminal, Web 1, Web 2)	Menu, IDE e endpoints `/api/*`	`compile_file` e as fases isoladas (tokens, AST, semântica, TAC, C++)
Analisador léxico	`Lexer(fonte).tokenize() → List[Token]`	Código-fonte
Analisador sintático	`Parser(tokens).parse() → Program`	Lista de tokens
Analisador semântico	`SemanticAnalyzer().analyze(programa) → bool` e `diagnostics`	AST
Gerador TAC	`TACGenerator().generate(programa, nome) → caminho do .tac`	AST validada

Componente	Interface fornecida	Interface requerida
Gerador C++	`CppCodeGenerator().generate(programa, nome) → bool`	AST validada, `cpp_template.py`, g++
Runtime de canais	`MiniParChannel`: `send`, `receive`	Sockets TCP (POSIX ou Winsock)
Runtime Fork/Join em núcleos	`__fork(núcleo, função) → tarefa`, `__join(tarefa) → valor`	`std::thread`, `promise/future`, afinidade do sistema operacional
Runtime Fork em host	`__fork_host<f>(ip, porta, args…) → tarefa` e o modo `--worker <porta>`	Serialização `MPValue`, sockets TCP, o mesmo programa no outro computador

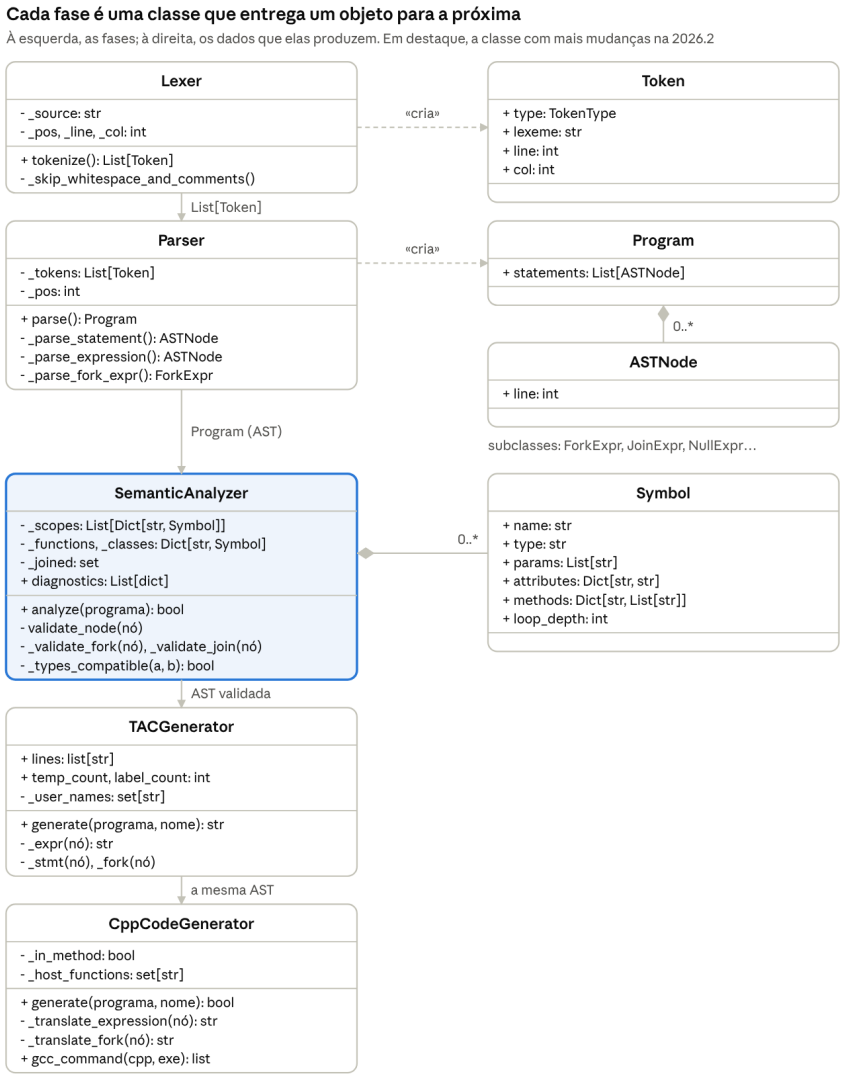
Fonte: Elaborado pelos autores (2026).

O `fork … on host` reaproveita o fork em núcleos: a chamada remota roda dentro de uma tarefa local comum, que envia a função e os argumentos ao worker e espera o resultado. Por isso `join` funciona igual para os três destinos. Cada função usada em `fork … on host` se registra no worker pelo nome e pelos tipos dos argumentos, então os dois computadores precisam rodar o mesmo programa compilado.

12 MODELAGEM DOS COMPONENTES: DIAGRAMA DE CLASSES

Cada componente da seção 11 corresponde a uma classe Python. As fases (à esquerda) não guardam estado entre compilações: são criadas, recebem a entrada, devolvem o resultado e são descartadas. Os dados (à direita) são `dataclasses` sem comportamento.

Figura 19 – Diagrama de classes: fases do compilador e dados que elas produzem



Fonte: Elaborado pelos autores (2026).

12.1 Nós da AST acrescentados na versão 2026.2

Todos os nós herdam de `ASTNode`, que guarda a linha do código-fonte. São 40 subclasses ao todo; as novas são:

Quadro 15 – Nós da AST acrescentados na versão 2026.2

Classe	Atributos	Significado
`ForkExpr`	`call: FuncCallExpr | MethodCall`, `destination: CoreTarget | HostTarget | None`	`fork f(x) [on destino]`; o valor da expressão é uma tarefa
`CoreTarget`	`index: ASTNode`	Destino `core(n)`

Classe	Atributos	Significado
`HostTarget`	`ip: ASTNode`, `port: ASTNode`	Destino `host(ip, porta)`
`JoinExpr`	`task: ASTNode`	`join t`; o valor é o retorno da subtarefa
`NullExpr`	—	Literal `null`

Fonte: Elaborado pelos autores (2026).

12.2 Classes do runtime C++

Quadro 16 – Classes e funções do runtime C++

Classe ou função	Responsabilidade
`MiniParChannel`	Canal TCP do `c_channel`: socket de escuta persistente e uma conexão nova por mensagem, com novas tentativas enquanto o outro lado não está pronto
`MiniParTask<R>`	Tarefa criada por `fork`: guarda a thread e o `future` com o resultado
`__fork(núcleo, função)` / `__join(tarefa)`	Cria a thread, fixa no núcleo pedido e devolve o resultado no `join`
`MPValue`	Valor serializável (número, texto, booleano, lista) usado no fork remoto
`__MPRemote<F, A…>`	Registro de cada função usada em `fork … on host`, pela chave `nome(tipos)`, para o worker saber o que executar
`__fork_host<F>(ip, porta, args…)`	Envia a chamada ao worker dentro de uma tarefa local e devolve o resultado ao `join`
`__mp_worker_from_args(argc, argv)`	Se o programa recebeu `--worker <porta>`, atende pedidos de fork em vez de rodar o `main`

Fonte: Elaborado pelos autores (2026).

13 TESTES DE INTEGRAÇÃO DOS COMPONENTES

Os testes de integração usam `pytest` e exercitam o pipeline inteiro: cada teste compila um programa MiniPar com os componentes reais (léxico → sintático → semântico → TAC → C++ → g++), executa o binário e confere a saída. Nos testes com dois computadores, cada computador é um processo separado na mesma máquina, conversando por TCP em `127.0.0.1`.

Comando, na raiz do projeto: `python -m pytest`

Resultado: 64 testes passando (mais 3 subtestes), em 3 min 18 s.

Quadro 17 – Suíte de testes automatizados

Arquivo	Testes	O que verifica	Autor
`tests/test_regressao.py`	50	Todo `.minipar` de `tests/` compila (30); 16 programas têm a saída comparada com `tests/expected_results/`; o TAC do Teste 7 é comparado com o esperado; o programa com erros de fork/join é rejeitado; o comando do g++ é montado certo para Linux e Windows	Pedro
`tests/test_fork_join.py`	3	Sobe um worker real e verifica fork para host junto com forks em núcleos, várias subtarefas no mesmo host e a mensagem de erro quando o worker está desligado	Anthony
`tests/test_dois_computadores.py`	4	Teste 1 original não trava; Teste 1 com cliente e servidor em processos separados; Teste 2 com Fibonacci no outro computador; Teste 8 com um lote no outro computador	Anthony
`tests/test_teste11.py`	6	Os 3 `.arm` gerados são iguais aos versionados; com assembler e emulador ARM disponíveis, os 3 montam, executam e deixam o resultado certo em `r8`	Anthony
`tests/test_benchmark_8_10.py`	1 (3 subtestes)	Testes 8, 9 e 10 paralelos: resultados iguais aos sequenciais e speedup informado	Vinicius

Fonte: Elaborado pelos autores (2026).

A comparação de saída ignora acentos e espaços em excesso e aceita tolerância numérica quando configurada (Teste 4, com pesos aleatórios: 0,05). Programas que leem do teclado recebem a entrada pelo dicionário `CONFIG` (por exemplo, o vetor do Teste 6). Para entrar na regressão, basta criar `tests/expected_results/<nome>.expected.txt`.

13.1 Defeitos encontrados pelos testes

Quadro 18 – Defeitos encontrados pelos testes

Defeito	Causa	Correção
Teste 1 original travava em 5 de 5 execuções	O `MiniParChannel` fechava a porta de recepção a cada `receive`, perdendo mensagens enviadas em sequência, e reaproveitava um socket que já tinha falhado	Porta de escuta aberta enquanto o canal existe e um socket novo a cada tentativa de envio
Teste 7 terminava com `malloc(): corrupted top size`	A pilha do programa era uma lista de 2 posições escrita por índice além do fim (stack[sp] com sp ≥ 2), corrompendo a memória	Pilha dinâmica com append e pop; acesso por índice fora dos limites agora gera erro de execução com mensagem
Falha de ligação no Linux	`-lws2_32` (Winsock) era passado ao g++ em todos os sistemas	Biblioteca e extensão `.exe` só no Windows

Defeito	Causa	Correção
`__floor` colidia com a glibc no Linux	Nome do builtin igual ao de uma função interna da biblioteca C	Renomeado para `__mp_floor`
Falso erro semântico em `i = n` com `n = len(…)`	O tipo de `len` não é conhecido na compilação e era tratado como incompatível	Tipos `var`/`unknown` passam a ser compatíveis com qualquer tipo
Métodos do parser do Teste 11 não compilavam em C++	Funções mutuamente recursivas com retorno `auto`	Inferência do tipo de retorno de funções que devolvem objetos (`std::shared_ptr<Classe>`)

Fonte: Elaborado pelos autores (2026).

14 TESTES DOS PROGRAMAS DE EXEMPLO 1 A 11

Todos os programas estão em `tests/` e foram executados com o compilador final. As saídas abaixo são reais, do ambiente Linux de desenvolvimento, salvo indicação.

Quadro 19 – Situação dos programas de teste 1 a 11

Teste	Programa	Recursos exercitados	Situação
1	`teste1_calculadora_cliente_servidor.minipar` e `dois_computadores/teste1_*`	Canais `c_channel`, `send`, `receive`, `input`	OK em um e em dois processos
2	`teste2_fatorial_fibonacci_par.minipar` e `dois_computadores/teste2_*`	Bloco `par` com dois blocos `seq`; na versão para dois computadores, fork on core e fork on host	OK
3	`teste3_neuronio.minipar`	Classes, objetos, laços, decimais	OK (regressão)
4	`teste4_rede_neural_xor.minipar`	Listas, `exp`, backpropagation	OK (regressão, tolerância 0,05)
5	`teste5_recomendacao.minipar`	Listas, funções, classes e rede neural	OK (regressão)
6	`teste6_quicksort_classes.minipar`	Classes, recursão, entrada do teclado	OK (regressão)
7	`teste7_quicksort_iterativo_tac.minipar`	Pilha explícita, TAC comparado com o esperado	OK (saída e TAC)
8	`teste8_sequencial`, `teste8_paralelo` e `dois_computadores/teste8_paralelo_host`	`fork`/`join`, `clock()`, um lote em outro computador	OK
9	`teste9_sequencial`, `teste9_paralelo`	`fork`/`join`, Mandelbrot em uma quinta tarefa	OK

Teste	Programa	Recursos exercitados	Situação
10	`teste10_sequencial`, `teste10_paralelo`	`fork`/`join`, Mandelbrot e Sierpinski	OK
11	`teste11/teste11_minicompilador.minipar`	Classes, `null`, concatenação, `write_file`, geração de ARMv7	OK, validado no CPULATOR

Fonte: Elaborado pelos autores (2026).

14.1 Teste 1: calculadora cliente-servidor

```
==== Calculadora cliente-servidor ====
Opcoes: +, -, *, /
Operacao (+,-,*,/): +
Operando 1: 3
Operando 2: 5
Servidor recebeu: + 3 5
Resultado: 8
```

Na versão para dois computadores, `teste1_servidor` e `teste1_cliente` rodam em processos (ou máquinas) diferentes, e o servidor registra `Servidor recebeu: 10 / 4 | resultado: 2.5`.

14.2 Teste 2: fatorial e Fibonacci em paralelo

```
Fibonacci:
0
1
```

```
1
2
3
5
```

```
8
13
Fatorial de 5: 120
```

14.3 Teste 3: neurônio simples (início e fim)

```
==== Teste 3: neuronio simples com
↪  classes e objetos ====
Entrada: 1 | Saida desejada: 0
#### Iteracao: 1
Peso: 0.5000
Saida: 1
Erro: -1
```

```
Peso do bias: 0.5000
…
#### Iteracao: 51
Peso: -0.0000
Saida: 0
Erro: 0
Parabens! O neuronio aprendeu.
```

14.4 Teste 4: rede neural XOR

```
Input: [0, 0], Predicted Output: 0.0068
Input: [0, 1], Predicted Output: 0.9865
Input: [1, 0], Predicted Output: 0.9893
Input: [1, 1], Predicted Output: 0.0140
```

14.5 Teste 5: recomendação de produtos

```
Produtos recomendados para
↪  voce:
Laptop
Tablet
```

```
Fones de ouvido
Camisa
Jaqueta
Sapatos
Geladeira
```

```
Maquina de lavar
Ar condicionado
Nao-ficcao
Ficcao cientifica
Fantasia
```

14.6 Teste 6: quicksort com classes (entrada `10 -3 -40 80 70 -100`)

```
Vetor original: [10, -3, -40, 80, 70, -100]
Vetor ordenado: [-100, -40, -3, 10, 70, 80]
```

14.7 Teste 7: quicksort iterativo

```
Array ordenado: [5, 12, 33, 61, 98]
```

14.8 Testes 8, 9 e 10: benchmark sequencial × paralelo

Cada programa avalia 160 atividades, em quatro lotes de 40, nas versões sequencial e paralela com `fork/join`. O Teste 9 acrescenta o fractal de Mandelbrot em uma quinta tarefa, e o Teste 10 acrescenta também o tapete de Sierpinski (grau 4, 81 × 81 caracteres). O checksum 4.854.810 é o mesmo nas duas versões de todos os testes.

Quadro 20 – Tempos e speedup dos Testes 8, 9 e 10

Teste	Sequencial (s)	Paralelo (s)	Speedup (i5-12450H, WSL2)	Speedup (Linux, 2 núcleos)	Speedup (Python do enunciado)
8	0,1156	0,0371	3,12×	1,77×	1,18×
9	0,1148	0,0357	3,21×	1,60×	1,02×
10	0,1104	0,0325	3,39×	1,83×	0,52×

Fonte: Elaborado pelos autores (2026); speedup dos programas Python conforme o enunciado do projeto.

O ganho maior que o do Python vem de dois fatores: o C++ gerado roda as threads de verdade em paralelo (o Python tem o GIL), e as cargas foram tornadas determinísticas, com o mesmo número de iterações por atividade, para que sequencial e paralelo façam exatamente o mesmo trabalho.

Saída do Teste 9 (paralelo):

```
Teste 9: benchmark sequencial e paralelo
Atividades: 160
Checksum sequencial: 4854810
Checksum paralelo: 4854810
Resultados: OK
Tempo sequencial (s): 0.114429
Tempo paralelo (s): 0.0716021
Speedup: 1.59812
Mandelbrot:
                .     #@@@@@@@@@@@@@@@@@@:
              .@:@@@- :@@@@@@@@@@@@@@@@@@:
           - .@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@.
           - .@@@@@@@@@@@@@@@@@@@@@@@@@@@@
              .@:@@@- :@@@@@@@@@@@@@@@@@@:
                 .     #@@@@@@@@@@@@@@@@@@:
```

Teste 8 com um lote em outro computador (`dois_computadores/teste8_paralelo_host.minipar`, Windows, worker em outro processo): Professor, Monitor 1 e Monitor 2 em `core(1)`, `core(2)` e `core(3)`; Monitor 3 em `host(ip, 5001)`.

```
Teste 8: benchmark sequencial e paralelo (3 nucleos + 1 computador remoto)
Atividades: 160
Professor (nucleo 1): 1032833
Monitor 1 (nucleo 2): 1159741
Monitor 2 (nucleo 3): 1236814
Monitor 3 (computador 127.0.0.1): 1425422
Checksum sequencial: 4854810
Checksum paralelo: 4854810
Resultados: OK
Speedup: 2.14655
```

No worker: `[WORKER] Executando avaliar_lote(i64,i64) pedida por 127.0.0.1`.

14.9 Teste 11: minicompilador de expressões para ARMv7

O Teste 11 é um compilador escrito em MiniPar: classes `Lexer`, `No` (nó da AST) e `MiniCompilador` transformam uma expressão aritmética em tokens, AST, código de três endereços e Assembly ARMv7, mostram a tabela de símbolos e o resultado e gravam um arquivo `.arm` com `write_file`. A saída da primeira expressão é igual à do programa Java fornecido pelo professor.

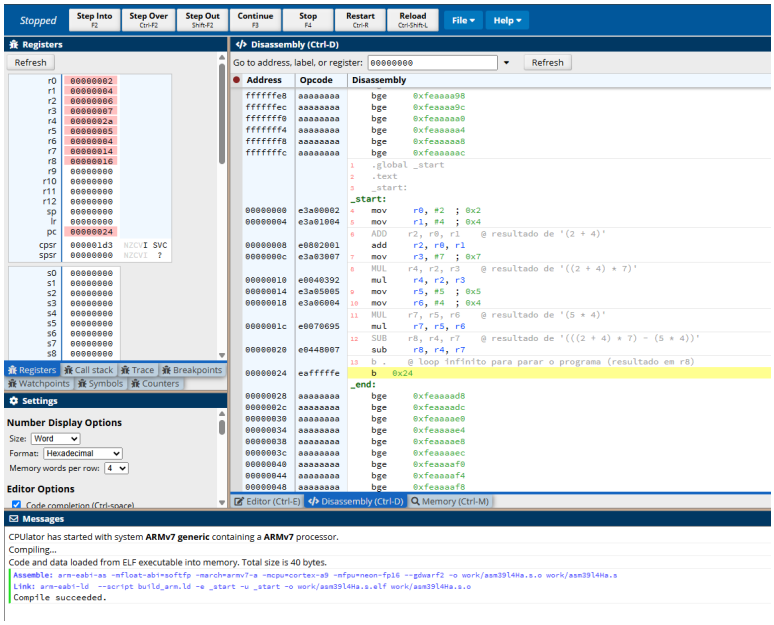
Quadro 21 – Resultados do Teste 11 no CPULATOR

Expressão	Resultado	Registros intermediários no CPULATOR	`r8` no CPULATOR
`((2+4)*7)-(5*4)`	22	r2 = 0x6, r4 = 0x2a (42), r7 = 0x14 (20)	`0x00000016` = 22
`((10 -8*7)+4)*2`	−84	r3 = 0x38 (56), r4 = 0xffffffd2 (−46), r6 = 0xffffffd6 (−42)	`0xffffffac` = −84
`((2+4)*7)-(5*4)`	22	iguais aos da expressão 1	`0x00000016` = 22

Fonte: Elaborado pelos autores (2026).

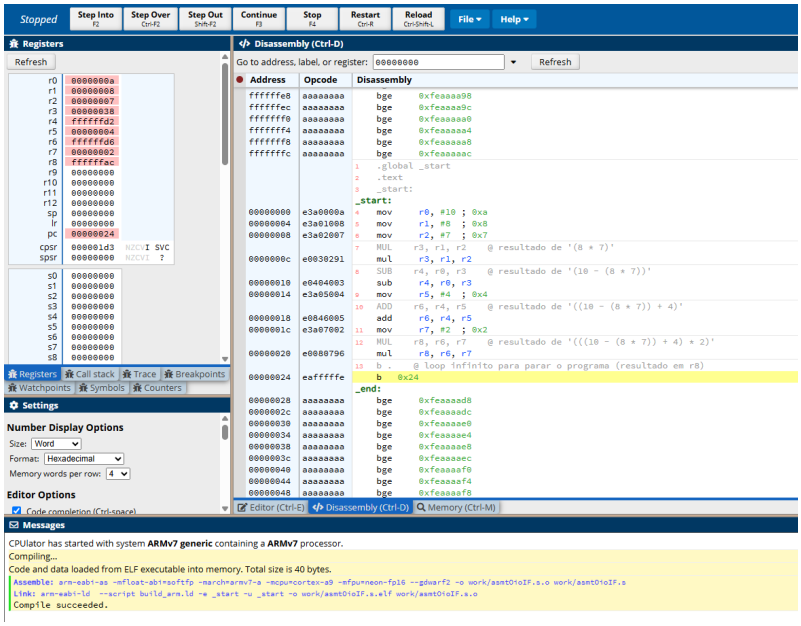
Os três `.arm` foram colados no CPULATOR (sistema ARMv7 generic), compilaram sem erro (40 bytes de código) e, executados até o laço final `b .`, deixaram o resultado em `r8`. O resultado negativo da expressão 2 aparece em complemento de dois. Os valores intermediários seguem o TAC gerado (t1 a t4).

Figura 20 – CPULATOR, expressão 1 `((2+4)*7)-(5*4)`: r8 = 0x00000016 (22)



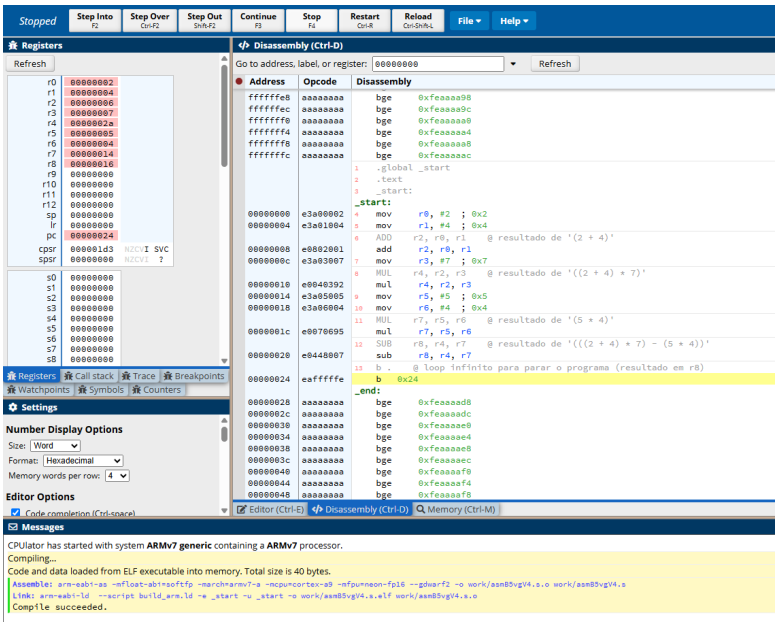
Fonte: Captura de tela do CPULATOR, elaborada pelos autores (2026).

Figura 21 – CPULATOR, expressão 2 `((10 -8*7)+4)*2`: r8 = 0xfffffac (–84)



Fonte: Captura de tela do CPULATOR, elaborada pelos autores (2026).

Figura 22 – CPULATOR, expressão 3 `((2+4)*7)-(5*4)`: r8 = 0x00000016 (22)



Fonte: Captura de tela do CPULATOR, elaborada pelos autores (2026).

15 IMPLEMENTAÇÃO

15.1 Tecnologias e ambiente

Quadro 22 – Tecnologias utilizadas

Tecnologia	Uso
Python 3 (testado com 3.13)	Todo o compilador: léxico, sintático, semântico, TAC e geração de C++
C++20 e g++ (`-std=c++20 -O3 -pthread`)	Código gerado e runtime (canais, fork/join, worker). Linux: g++ 13; Windows: MinGW-w64 com `-lws2_32`
`std::thread`, `std::promise`/`std::future`	Subtarefas do `fork` e resultado do `join`
Afinidade de CPU (`pthread_setaffinity_np` no Linux, `SetThreadAffinityMask` no Windows)	Fixar a subtarefa no núcleo de `core(n)`
Sockets TCP (POSIX e Winsock)	Canais e `fork … on host`
Flask 3.0	Back-end das interfaces Web 1 e Web 2
CodeMirror 5.65 e D3 7.8	Editor de código e árvore da AST na Web 2
pytest 8	Testes de regressão e de integração
CPULATOR (ARMv7 generic)	Execução dos `.arm` gerados pelo Teste 11
`arm-linux-gnueabihf-as` e `qemu-arm`	Validação automática dos `.arm` no pytest (opcional)
Git e GitHub	Versionamento, uma branch por integrante e pull requests para o `main`

Fonte: Elaborado pelos autores (2026).

O grupo trabalhou em Windows e Linux (incluindo WSL2). O mesmo código-fonte do compilador funciona nos dois: só o comando do g++ muda, e isso fica isolado em `CppCodeGenerator.gcc_command`.

15.2 Passo a passo das execuções

1. Instalação

```
git clone https://github.com/anthonygpm/Projeto-MiniPar-2026.2.git
cd Projeto-MiniPar-2026.2
pip install -r requeriments.txt
g++ --version          # precisa de suporte a C++20
```

2. Compilar e executar um programa pelo terminal

```
python main.py tests/teste8_paralelo.minipar
./output/teste8_paralelo            # Linux
.\output\teste8_paralelo.exe        # Windows (PowerShell)
```

O compilador mostra as quatro fases e grava em `output/` o `.tac`, o `.cpp` e o executável. Sem argumentos, `python main.py` abre a escolha entre o menu do terminal e a interface Web.

3. Interfaces Web

`python main.py`, opção 2, e abrir `http://127.0.0.1:5000/` (Web 1) ou `http://127.0.0.1:5000/didatico` (Web 2).

4. Fork em outro computador (dois computadores na mesma rede)

1. Compilar o mesmo programa nos dois computadores (ex.: `python main.py tests/fork_join/demo_host.minipar`).
2. No computador 2, liberar a porta no firewall (Windows, PowerShell como administrador): `New-NetFirewallRule -DisplayName "MiniPar worker" -Direction Inbound -Protocol TCP -LocalPort 5001 -Action Allow`.
3. No computador 2, iniciar o worker: `.\output\demo_host.exe --worker 5001`.
4. No computador 1, conferir a conexão (`Test-NetConnection <ip> -Port 5001`) e executar `.\output\demo_host.exe`, informando o IP do computador 2.

Execução real, computador 1 (Anthony, 192.168.151.41) e computador 2 (Pedro, 192.168.151.118):

```
PS> .\output\demo_host.exe
IP do computador 2 (Enter = 127.0.0.1): 192.168.151.118
Total das 4 subtarefas: 1599999996
[Monitor remoto] Concluiu: Aluno 1
[Monitor remoto] Concluiu: Aluno 2
[Monitor remoto] Concluiu: Aluno 3
Tempo: 0.38 s
```

O resultado é o mesmo da execução com os dois papéis no mesmo computador. No log do worker, cada subtarefa aparece como `[WORKER] Executando carga(i64,i64) pedida por <ip do computador 1>`.

5. Testes 1 e 2 em dois computadores

O servidor (`tests/dois_computadores/teste1_servidor.minipar`) roda em um computador e o cliente (`teste1_cliente.minipar`) em outro; cada um pergunta o IP do outro ao iniciar. O Teste 2 (`teste2_dois_computadores.minipar`) envia o cálculo de Fibonacci ao worker na porta 5001.

6. Teste 11 no CPULATOR

1. `python main.py tests/teste11/teste11_minicompilador.minipar` e executar o binário, apertando Enter para usar as 3 expressões do enunciado.
2. Abrir cpulator.01xz.net, escolher o sistema ARMv7 generic e colar o conteúdo de `tests/teste11/arm/teste11_expressaoN.arm` (os mesmos arquivos ficam em `output/`).
3. Compile and Load (F5), Continue (F3) e conferir `r8` quando o programa chega ao laço final.

7. Suíte de testes

`python -m pytest` (todos) ou `python -m pytest -k teste4` (um só).

15.3 Problemas encontrados e decisões

Quadro 23 – Problemas encontrados e decisões

Problema	Solução ou decisão
Firewall do Windows bloqueava o worker	Regra de entrada para a porta 5001, criada em um PowerShell de administrador
Executável do MinGW fechava sem mensagem com o Anaconda ativo	O ambiente `(base)` põe no PATH uma `libstdc++-6.dll` antiga; usar `conda deactivate` antes de executar
`ping` falhava entre os computadores	O Windows bloqueia ICMP por padrão; o teste certo é `Test-NetConnection <ip> -Port 5001`
`core(n)` deu speedup menor que sem afinidade em um processador híbrido	Ver abaixo

Fonte: Elaborado pelos autores (2026).

Afinidade em processador híbrido. No notebook do Anthony (Intel híbrido, 12 núcleos físicos e 14 lógicos: 2 núcleos de desempenho com Hyper-Threading e 10 de eficiência), o programa `demo_paralelo` com 4 subtarefas teve:

Quadro 24 – Speedup do demo_paralelo com e sem afinidade de núcleo

Variante	Sequencial	Paralelo	Speedup
`fork carga(k, N) on core(k + 1)`	0,48 s	0,27 s	1,75×
`fork carga(k, N)` (sem `on`)	0,48 s	0,20 s	2,46×

Fonte: Elaborado pelos autores (2026).

O runtime fixa `core(n)` no processador **lógico** n−1. Nesse processador, os lógicos 0 a 3 são os 2 núcleos de desempenho, cada um com 2 threads, então `core(1)` a `core(4)` dividiram apenas 2 núcleos físicos. Sem afinidade, o sistema operacional espalha as threads entre os núcleos de desempenho e de eficiência, e o ganho sobe. Núcleo lógico, núcleo físico e tipo de núcleo não são a mesma coisa. A melhoria possível, mapear `core(n)` para o n-ésimo núcleo físico com os de desempenho primeiro, foi deixada de fora por falta de tempo (item PB14 do backlog). Em uma máquina sem núcleos híbridos, a afinidade funciona como esperado: no ambiente Linux de desenvolvimento, com 2 núcleos, o mesmo programa teve speedup de 1,86×.

16 CÓDIGO-FONTE, TESTES E RELATÓRIO NO GITHUB

Repositório: github.com/anthonygpm/Projeto-MiniPar-2026.2

Quadro 25 – Organização do repositório

Pasta ou arquivo	Conteúdo
`src/`	Compilador: `lexer.py`, `parser.py`, `ast_nodes.py`, `semantic.py`, `tac.py`, `codegen.py`, `cpp_template.py` (runtime C++)
`main.py`, `menu.py`, `test_orchestrator.py`	Ponto de entrada, menu do terminal e sessões de teste com dois terminais
`app.py`, `web_pipeline.py`, `templates/`, `static/`	Interfaces Web 1 e Web 2
`tests/`	Programas dos Testes 1 a 11, exemplos de fork/join, versões para dois computadores, saídas esperadas e suíte pytest
`BNF-grammar.md`	Gramática da linguagem
`output/`	Arquivos gerados (`.tac`, `.cpp`, executáveis e `.arm` do Teste 11)

Fonte: Elaborado pelos autores (2026).

17 CONCLUSÃO

A versão 2026.2 do compilador MiniPar acrescentou paralelismo explícito à linguagem: `fork` dispara uma função como subtarefa em um núcleo escolhido, em um núcleo escolhido pelo sistema operacional ou em outro computador, e `join` espera a subtarefa e devolve o seu resultado. As três formas compartilham o mesmo modelo de tarefa no runtime C++, então o programador muda apenas o destino depois de `on`. O mecanismo foi validado entre dois computadores reais na mesma rede, com o mesmo resultado da execução local.

Os Testes 1 a 11 do enunciado rodam com o compilador final. Nos Testes 8, 9 e 10, as versões paralelas obtiveram speedup entre 1,6× e 3,4×, conforme a máquina, contra 0,5× a 1,2× dos programas Python de referência. O Teste 11, um compilador de expressões escrito na própria MiniPar, exigiu `null`, concatenação de texto com número, `write_file` e tipos de retorno de objetos, e produziu Assembly ARMv7 que executa corretamente no CPULATOR.

A interface didática (Web 2) mostra cada fase do compilador para o programa do próprio aluno. A suíte de 64 testes automatizados encontrou defeitos reais que não apareciam em execuções manuais, como o travamento dos canais no Teste 1.

Como trabalho futuro, ficam o mapeamento de `core(n)` para núcleos físicos em processadores híbridos, o envio de objetos para outro computador (hoje apenas números, textos, booleanos e listas) e a descoberta automática de workers na rede.

REFERÊNCIAS

AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. **Compiladores**: princípios, técnicas e ferramentas. 2. ed. São Paulo: Pearson Addison-Wesley, 2008.

BOSTOCK, Mike. **D3.js**: data-driven documents. Versão 7.8. [S. l.], 2023. Disponível em: https://d3js.org/. Acesso em: 10 nov. 2026.

CPULATOR. **CPUlator computer system simulator**: ARMv7 generic. [S. l.], [20–]. Disponível em: https://cpulator.01xz.net/. Acesso em: 10 nov. 2026.

DOMINGUEZ, Arturo Hernandez. **Elementos de implementação utilizando componentes**. Material de apoio da disciplina de Compiladores. Maceió: Universidade Federal de Alagoas, Instituto de Computação, [202-].

DOMINGUEZ, Arturo Hernandez. **Modelagem do interpretador usando componentes de software**. Material de apoio da disciplina de Compiladores. Maceió: Universidade Federal de Alagoas, Instituto de Computação, 2025.

DOMINGUEZ, Arturo Hernandez. **Projeto compilador MiniPar 2026.2**. Especificação do projeto da disciplina de Compiladores. Maceió: Universidade Federal de Alagoas, Instituto de Computação, 2026.

GUALANDI, Hugo Musso; IERUSALIMSCHY, Roberto. A surprisingly simple Lua compiler. *In*: BRAZILIAN SYMPOSIUM ON PROGRAMMING LANGUAGES, 25., 2021, Joinville. **Proceedings** […]. New York: ACM, 2021. DOI: 10.1145/3475061.3475077.

HAVERBEKE, Marijn. **CodeMirror 5**: in-browser code editor. [S. l.], 2023. Disponível em: https://codemirror.net/5/. Acesso em: 10 nov. 2026.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 14882:2020**: programming languages — C++. Genebra: ISO, 2020.

LIMA, Emanuele Vitória de Jesus; QUINTELA, Jose Lucas de Oliveira; BOMFIM, Luiz Miguel de Melo. **MiniPar-Improvement**: compilador da linguagem MiniPar 2026.1. Maceió, 2026. Disponível em: https://github.com/luizwhirl/minipar-improvement. Acesso em: 10 nov. 2026.

PALLETS PROJECTS. **Flask documentation (3.0.x)**. [S. l.], 2023. Disponível em: https://flask.palletsprojects.com/. Acesso em: 10 nov. 2026.

PYTEST-DEV. **pytest documentation**. [S. l.], 2024. Disponível em: https://docs.pytest.org/. Acesso em: 10 nov. 2026.

PYTHON SOFTWARE FOUNDATION. **The Python language reference**. Versão 3.13. [S. l.], 2024. Disponível em: https://docs.python.org/3/reference/. Acesso em: 10 nov. 2026.